**The School of Mathematics**

THE UNIVERSITY
*of* EDINBURGH

# Scored-based generative models and portfolio generation

**by**

**Pavel Maniakin**

Dissertation Presented for the Degree of
MSc in Financial Mathematics

August 2025

Supervised by
Sotirios Sabanis

# Contents

**1 Introduction** **2**

**2 Literature Review** **3**

**3 Methodology** **8**

3.1 Algorithm SGM (Bruno et al., 2025) . . . . . 8

3.2 Convergence in terms of the Wasserstein-2 distance for SGMs . . . . . 12

3.3 Algorithm SBTS (Hamdouche & Pham, 2023) . . . . . 14

**4 Results** **17**

4.1 One-Dimensional Gaussian Process: Analytical Score and Estimation of $\theta$ (Appendix 1) . . . . . 17

4.2 Estimating $\theta$ with a neural network (Appendix 1) . . . . . 20

4.3 Two-dimensional Gaussian distribution (Appendix 2) . . . . . 22

4.4 One-dimensional exponential distribution (Appendix 3, 4) . . . . . 23

4.5 One-dimensional Mixture of Gaussian (Appendix 5) . . . . . 25

4.6 Two-dimensional Mixture of Gaussian (Appendix 6) . . . . . 27

4.7 Portfolio generation using SGMs (Appendix 7, 8, 9, 10, 11) . . . . . 29

4.8 Results Schrödinger Bridge Algorithm 3.1 (Appendix 12, 13, 14, 15, 16) . . 33

# Abstract

This research illustrates score-based generative models (SGMs) by a two-stage approach designed not only to develop a solid theoretical foundation but also to demonstrate their practical value in finance. The research has been based on existing work that provides mathematical formulas and convergence guarantees, starting with focusing on a setting where the score function is analytically known. This controlled environment allows for detailed verification of the training process and optimization of the model parameters. The theory is then extended to more realistic cases where the score function must be approximated using a neural network in order to be able to apply it into a real-world financial time series.

For testing practical relevance, the proposed OU–SGLD-based SGM is compared with the Schrödinger Bridge method for time series generation. The test is conducted in the context of synthetic portfolio construction, where our method is more profitable in four out of five market scenarios. This dual focus — theoretical precision combined with demonstrated financial performance — highlights the potential of SGMs as both a mathematically interpretable and profit-oriented tool.


# 1 Introduction

Score-based generative models (SGMs) (Song & Ermon, 2019; Song & Sohl-Dickstein, et al., 2021), developed over the past few years became a main area in probabilistic generative modelling. Their use in image synthesis (Dhariwal & Nichol, 2021; Rombach et al., 2022), audio generation (Kong et al., 2021), along with solving inverse problems (Chung et al., 2024) showed a great success. This led to more interest in more rigorous theoretical analysis (De Bortoli et al., 2021; Chen & Theodorou., 2023; Conforti et al., 2024; Bruno et al., 2025). In a great numbers of applications, SGMs have performed good and moreover outperforming other frameworks such as generative adversarial networks (GANs), variational autoencoders (VAEs), and normalizing flows - this happens partly because of their resistance in problems like mode collapse (Salmona et al., 2022).

The main idea behind SGMs is that they combine two stochastic processes: a forward diffusion process that changes a complex data distribution into Gaussian noise, and a backward process that rebuilds data from noise using the score-function, the gradient of the log-density at time $t$. Since the score depends on the unknown data distribution, it must be approximated using score matching approaches that leads to combination the

optimization and sampling procedures (Bruno et al., 2025).

In this work, we focus on SGMs that use the Ornstein–Uhlenbeck (OU) process as the forward SDE. The OU process provides analytical understanding, Gaussian transition kernels, and closed-form expressions for intermediate densities. These properties enable the score function to be explicitly calculated in certain cases, providing a useful benchmark for verifying numerical methods and studying convergence. For score approximation, we use as a foundation Stochastic Gradient Langevin Dynamics (SGLD), which — as shown in Bruno et al., 2025 — allows explicit non-asymptotic convergence rates in the Wasserstein-2 distance metric under certain assumptions on the data distribution.

In addition to our primary OU–SGLD framework, the results are compared with an another approach for time-series generation — the Schrödinger Bridge method Hamdouche & Pham, 2023. Unlike SGMs, the Schrödinger Bridge method sets up the problem as a random interpolation between prescribed initial and terminal distributions. In our case, it allows us to compare the profitability of synthetic financial portfolios.

Overall, this work combine a rigorous theoretical framework — inspired by the analysis of Bruno et al., 2025 — with applied experiments on financial time series. The study begins with going through the cases that are easy to analyze, those with known score functions to check our training procedure, and then moves to more complex scenarios where the score must be estimated by a neural network. The study ends with a comparative financial review against the Schrödinger Bridge approach, focusing on portfolio return metrics.

# 2 Literature Review

The idea of score-based generative models (SGMs) was first proposed by Song & Ermon, 2019 and later continued in the work of Song & Sohl-Dickstein, et al., 2021, where a new method was formulated using stochastic differential equations (SDEs), which allows achieving good results in image synthesis tasks. These studies made the foundation for linking the score-function to the gradient of the log-density of the data distribution at a given noise level, as well as to the drift term of the reverse-time SDE. Subsequent experiments demonstrated that SGMs remain stable even in high-dimensional settings, provided that the score function is accurately approximated.

The development of score-based generative models (SGMs) began with the work of Song & Ermon, 2019, where the concept of data generation through the integration of a reverse stochastic differential equation (SDE) was first introduced. In this formulation, the drift term depends on the so-called score function, defined as the gradient of the logarithm of the data distribution's density at each noise level. This work laid the foundation for subsequent advances and investigations into the method. Later, (Song & Sohl-Dickstein,

et al., 2021) expanded the approach by proposing an entire family of forward and reverse processes, including variations in the noise schedule and the choice of SDE formulation. Their findings showed that SGMs can perform better than GANs and VAEs in image generation tasks if the score function is accurately approximated. They can also avoid problems like mode collapse, which occurs when a generative model only generate a small number of typical examples and fail to capture the all aspects of the target distribution, while remaining stable even in high-dimensional settings. This issue is particular to GANs, where the discriminator and generator reach a local equilibrium where the generator obtains from reproducing only a small number of readily identifiable patterns, causing the model to become "trapped" in a limited subset of the data. In SGMs, this effect is largely mitigated because the generation process is formulated as stochastic integration of the reverse SDE with a deterministic score-function drift and the addition of noise. As a result, at each step the model incorporates information about the structure of the data density and gradually “unfolds” the noise back into data, rather than attempting to “guess” the entire object in a single update, as is the case in GANs.

With the growing interest in SGMs, a number of works with a strong theoretical focus have emerged. For example, Chen & Theodorou., 2023 investigated non-asymptotic bounds in the KL-divergence and total variation (TV) metrics, concentrating on the relationship between the accuracy of the score-function approximation and the quality of the resulting generation. The authors showed in their study that a polynomial convergence rate with respect to dimensionality can be achieved under certain assumptions about the regularity and smoothness of the data distribution, namely the boundedness of its growth for large norms and the Lipschitz continuity of the score function. Since the dimensionality of the data in real-world applications can reach hundreds or more, this property is crucial because any exponential dependence would make the method unfeasible. Furthermore, the authors examined both continuous and discrete numerical schemes for integrating the reverse process, demonstrating that the intended convergence rate can be maintained by adjusting the integration step size. These findings are limited in their applicability in fields like finance or bioinformatics, where heavy-tailed distributions are common, because they only hold under certain distributional assumptions, most notably strong log-concavity and sub-Gaussian tails.

The research direction started by Chen & Theodorou., 2023 was expanded in the work of Conforti et al., 2024, with an emphasis on easing the assumptions introduced on the data. They proposed a new assumption of finite relative Fisher information with respect to a Gaussian distribution in place of the rather restrictive requirement of strong log-concavity (SLC), which is rarely met in real-world scenarios. A critical measure of the smoothness and structural regularity of the data density in this scenario is the Fisher information; if it is finite, it indicates that the distribution is not overly "jagged" or

defined by sharp spikes. This point of view is important because it grounds the analysis in the well-researched Gaussian distribution, which is a common "terminal state" for many diffusion processes. Their work is especially noteworthy because, under this assumption, they were able to obtain non-asymptotic KL-divergence bounds that preserve polynomial dependence on dimensionality, apply to a wider class of distributions (including those with heavier than Gaussian tails), and do not require strong log-concavity. The scope of rigorous theoretical guarantees from idealized distributions to more realistic data settings is successfully expanded by these concepts. However, the method has drawbacks of its own. For instance, the bounds still require for a certain degree of smoothness of the score function, controlled growth at infinity, and the capacity to adequately approximate the score function. Such requirements may not be fully satisfied in real-world applications where neural networks are used to approximate the score, which could result in a decline in generative quality compared to theoretical predictions.

These works are definitely very important, but they impose some limitations that are rarely met in real-world situations, especially in the financial industry. The Bruno et al., 2025 study is notable for this exact reason. The authors of this paper examined the situation in which the forward process is a linear Orstein-Uhlenbeck (OU) SDE, which admits transition densities with a closed-form expression. They introduced an auxiliary Gaussian process with a known analytical density that coincides with the true forward process at the initial and intermediate time slices. This construction makes it possible to control the score-function approximation error in the $L^2$ norm with respect to the auxiliary distribution, rather than directly with respect to the true data distribution (which is generally unknown). Building on this approach, they derived non-asymptotic convergence bounds in the Wasserstein-2 metric. In the purely Gaussian setting, these bounds show that the generation error grows with dimension no faster than $d$, where $d$ denotes the data-space dimensionality—for example, the number of assets multiplied by the number of time points in a financial time series. This result is essentially optimal, and further improvement in the convergence rate is, at present, not possible. For more complex scenarios, in which the score function is unknown and approximated using stochastic gradient Langevin dynamics (SGLD), the authors obtained formulas that explicitly link the generative quality to specific patterns of the algorithm. They demonstrated that certain parameters are critical. For instance, the integration step size in the reverse process plays a dual role: too small a step slows down computation but yields a more accurate reproduction of the dynamics, whereas too large a step introduces accumulated numerical error that directly propagates into the final distribution. Furthermore, the number of SGLD iterations determines how well the structure of the score function can be captured at each noise level. An excessive number of iterations might be unnecessary if the noise already smooths out the finer details of the data, whereas a small number produces a coarse approximation. The amount of noise introduced into the SGLD is

equally crucial and needs to be carefully balanced. Noise is necessary for exploration of the space and for avoiding entrapment in local minima, but excessive noise blurs the score estimate, introducing variability into the reverse process.

The particular value of Bruno et al., 2025 lies in the fact that they did not treat all of the above-mentioned parameters in isolation. Instead, they incorporated them into a unified analytical framework that accounts for the accumulation of error throughout the entire reverse process. This is crucial because, in practical SGMs, each score-approximation error at an early step may appear negligible on its own, yet in multi-step integration it propagates forward, amplifies, and ultimately determines the quality of the final output.

To validate this, the authors conducted a series of numerical experiments in which they varied the integration step size, the length of the SGLD trajectories, and the noise level, while monitoring changes in the Wasserstein-2 distance between the generated and target distributions. The results were intuitive: careful tuning of these parameters led to a sharp reduction in the Wasserstein-2 distance, whereas coarse tuning resulted in a noticeable increase in divergence.

In parallel, an alternative theoretical framework known as the Schrödinger Bridge (SB) was also evolving. This method comes from stochastic analysis and is based on the idea of finding the most likely stochastic process that connects two given distributions at the start and end points. De Bortoli et al., 2021 showed that the Schrödinger Bridge (SB) framework follows this logic exactly. This is very helpful when data is missing or when strict physical or economic rules had to be followed. For instance, in dynamic modeling problems where the starting and ending states are known but the path in between needs to be rebuilt, SB gives a formal way to set conditional probabilities and lower the "cost" of not following the rules.

In the paper Hamdouche & Pham, 2023 they took things a step further by using SB directly to create time series, even financial ones. The initial distribution is based on data from the past, whereas the terminal distribution is based on the time in the future that needs to be modeled. This method allows the model to create intricate temporal patterns, such as correlations and autocorrelations characteristic of five complex markets. Their tests showed that the method can not only make time series look like the originals but it can also keep important statistical properties like volatility and return distributions. But this method also has its limits. For example, the Schrödinger Bridge (SB) does not guarantee strict convergence under heavy-tailed distributions or unstable dynamics, which are conditions that are very common in financial markets. This means that the SB method can reproduce the structure of the data that was seen, but its behavior in unusual or extreme situations may be harder to predict.

There have also been studies that have tried to relax data requirements or change the

assumptions about the score function in order to make theoretical guarantees for SGMs more useful. The example of such work was developed by Yang & Wibisono, 2023. They concentrated on the case where the score-approximation error only has sub-Gaussian tails, not strict boundedness or exponential decay. This is an important point to keep mind because, as has already been said, real-world data often has heavy tails. Their results showed that even under such milder conditions, it is still possible to control the convergence of generation—albeit with slower rates than in more classical theories. Nonetheless, their framework does not provide direct guarantees for cases with pronounced extremal behavior in the data.

At the same time, the work of Wenliang & Moran, 2023 focused on the observation that many complex and formally high-dimensional datasets, in reality, occupy only a small region within the full space of possible states. For example, in the case of financial portfolios comprising dozens of assets, the feasible combinations of prices and returns are not distributed across the entire space but rather concentrated on a much lower-dimensional "surface" — a so-called subspace. If this subspace can be reliably identified and estimated in advance, both training and generation can be restricted to it. This approach significantly reduces computational cost and lowers the risk of overfitting; however, it carries the danger that, if the estimated subspace is inaccurate, important structural details of the data may be lost, thereby undermining the fidelity of the generated samples.

Finally, Benton et al., 2024 proposed a technique for early stopping in the generative iterations. They concentrated on determining the optimal stopping point to achieve nearly linear convergence in KL-divergence with respect to dimensionality, which sharply reduces the computational cost of generation in large-scale settings. However, this method leaves open the question of long-term stability: stopping too early risks under-reconstructing the data, whereas stopping too late may lead to the accumulation of numerical integration errors and score-approximation inaccuracies.

Thus, the literature in the SGM domain can broadly be divided into two categories: on the one hand, the pursuit of rigorous theoretical guarantees; on the other, the need to adapt methods to complex, unstable, and often high-dimensional data typical of real-world scenarios. My work bridges these two directions: in the first stage, I will rely on a strictly defined setting with synthetic data, where the score function is known analytically, allowing for precise verification of the model implementation. Subsequently, the entire framework will be transferred to more realistic financial conditions, where the score is approximated by a neural network trained using SGLD, and later compared against an alternative method — namely, the Schrödinger Bridge (SB).

# 3 Methodology

## 3.1 Algorithm SGM (Bruno et al., 2025)

In many studies on diffusion-based generative models, the core idea lies in the gradual noising of data—namely, transforming the original complex distribution into a simpler one, typically the standard Gaussian. For this purpose, my work employs the Ornstein–Uhlenbeck (OU) process, distinguished by its analytically tractable dynamics and closed-form expressions for transition distributions. This approach is described in greater detail in Bruno et al. (2025). The OU process can be interpreted as a stochastic motion that continuously pulls the system back toward zero while simultaneously adding Gaussian noise. This leads to the following equation:

$$dX_t = -X_t\,dt + \sqrt{2}\,dB_t, \quad X_0 \sim \pi_D \tag{1}$$

where:

- $(B_t)_{t\in[0,T]}$ is a standard Brownian motion in $\mathbb{R}^M$;
- $X_0$ is a random variable with the **initial data distribution** $\pi_D$;
- the coefficients are chosen so that the invariant distribution is the standard Gaussian $\mathcal{N}(0, I_M)$.

This form is crucial for the present study because we know that the generation error depends solely on the accuracy of the score-function approximation and on the parameters of the reverse-time integration. There are no additional complexities arising from the data structure—everything is fully controlled. Moreover, the distribution of $X_t$ is known in closed form for any $t$, and the exponential decay of the drift enables a smooth transition from the original data to pure noise.

This process is linear and admits an exact analytical solution:

$$X_t = e^{-t}X_0 + \sqrt{1-e^{-2t}}\,Z_t, \quad Z_t \sim \mathcal{N}(0, I_M). \tag{2}$$

- The original signal $X_0$ decays with the coefficient $e^{-t}$;
- Gaussian noise with variance $\sqrt{1-e^{-2t}}$ is added, increasing over time;
- As $t \to \infty$, the influence of the original data vanishes, and $X_t$ is distributed as $\mathcal{N}(0, I_M)$.

The intuition behind this process is that at time $t = 0$ we have clean data, and as $t$ increases, noise is gradually added. By the end (for large $T$), we obtain almost independent Gaussian noise, which can be easily sampled.

The idea of the reverse process is to invert the action of the forward Ornstein–Uhlenbeck SDE, where the data $X_0 \sim \pi_D$ are progressively corrupted with noise, transforming them into Gaussian samples $X_T \sim \mathcal{N}(0, I_M)$. The reverse process should take noise $\mathcal{N}(0, I_M)$ and, moving backward in time, reconstruct data distributed according to $\pi_D$.

If the density of the forward process at time $t$ is denoted by $p_t$, then the theorem of Haussmann–Pardoux (1986) states that the reverse process $(Y_t)_{t\in[0,T]}$ is described by the SDE:

$$dY_t = (Y_t + 2\nabla \log p_{T-t}(Y_t))\, dt + \sqrt{2}\, d\bar{B}_t, \quad Y_0 \sim \mathcal{L}(X_T). \tag{3}$$

Here:

- $\nabla \log p_{T-t}(x)$ is the *score function*, which means the gradient of the log-density at time $T - t$, where, $p_t(x)$ denotes the probability density of the state $X_t$ at time $t$, and $\nabla$ is the gradient with respect to the spatial variable $x$. Thus, $\nabla \log p_t(x)$ is a vector pointing in the direction of the steepest increase of the log-density at point $x$. ;
- $\bar{B}_t$ is an independent Brownian motion;
- $Y_0$ starts from the distribution coinciding with the terminal distribution of the forward process.

In our case, $Y_t := X_{T-t}$, and therefore, as the initial condition of the reverse process, we take

$$\widetilde{Y}_0 \sim \pi_\infty = \mathcal{N}(0, I_M),$$

which yields the simplified form:

$$d\widetilde{Y}_t = \left(\widetilde{Y}_t + 2\nabla \log p_{T-t}(\widetilde{Y}_t)\right) dt + \sqrt{2}\, d\bar{B}_t. \tag{4}$$

To initiate the reverse process, it is necessary to know $\nabla \log p_{T-t}(x)$ for all $t$ and $x$. However, in real-world scenarios, this poses a challenge, as the data distribution $\pi_D$ is unknown, and thus the score function cannot be exactly calculated. To overcome this limitation, we propose training an approximation of the score function, $s(t, x, \theta) \approx \nabla \log p_t(x)$, via *score matching* — an optimization procedure that minimizes the discrepancy between the predicted and the true score.

The intuition behind the reverse process is that $\widetilde{Y}_t$ in the drift term "pulls" the trajectory from noise toward the center, while the term $2\nabla \log p_{T-t}$ incorporates knowledge of the

structure of the original data. The noise term $\sqrt{2}\,d\widetilde{B}_t$ is necessary to ensure that the trajectory is not strictly deterministic - this preserves the variability in generation. If the score approximation is accurate, then as we integrate "backwards" from pure noise, we progressively approach $\pi_D$.

In the context of score matching — particularly in real-world applications — it is generally impossible to compute the exact score function $\nabla \log p_t(x)$, since the density $p_t$ is unknown. Instead, we construct an approximator $s(t, x, \theta)$, implemented as a neural network, where $\theta$ denotes the vector of its weights. The training then consists of selecting $\theta$ so as to minimize the score matching functional Bruno et al., 2025.

In discrete form, the optimization problem is given by

$$\min_{\theta \in \mathbb{R}^d} U(\theta) := \int_{\varepsilon}^{T} \frac{\kappa(t)}{T-\varepsilon} \int_{\mathbb{R}^M} \left|\nabla \log p_t(x) - s(t,\theta,x)\right|^2 p_t(x)\,dx\,dt, \tag{5}$$

where $\kappa(t)$ is a weighting function, and $\varepsilon > 0$ is introduced to avoid degeneracy at $t = 0$.

$$U(\theta) = \mathbb{E}\left[\kappa(\tau)\left|\sigma_\tau^{-1} Z + s\left(\tau, \theta, m_\tau X_0 + \sigma_\tau Z\right)\right|^2\right] + C, \tag{6}$$

where $\tau \sim \text{Uniform}([\varepsilon, T])$, $Z \sim \mathcal{N}(0, I_M)$, $m_\tau = e^{-\tau}$, $\sigma_\tau = \sqrt{1 - e^{-2\tau}}$, and the constant $C$ does not depend on $\theta$.

It is also worth noting that, in this formulation, we directly employ the approximation of the score function, which is precisely what is useful for our study. However, we aim to eliminate the constant term so that we can focus on minimizing the stochastic gradient of the function $U(\theta)$.

$$H(\theta, \mathbf{x}) = 2\kappa(t) \sum_{i=1}^{M} \left(\sigma_t^{-1} z^{(i)} + s^{(i)}(t, \theta, m_t x_0 + \sigma_t z)\right) \nabla_\theta s^{(i)}(t, \theta, m_t x_0 + \sigma_t z), \tag{7}$$

where $\mathbf{x} = (t, x_0, z) \in \mathbb{R}^m$ and $m = 2M + 1$.

Here:

- $t \in [0, T]$ — time, chosen uniformly from the interval $[0, T]$;
- $x_0 \in \mathbb{R}^M$ — a sample from the initial data distribution $\pi_D$;
- $z \in \mathbb{R}^M$ — standard Gaussian noise $\mathcal{N}(0, I_M)$.

After this, SGLD (Stochastic Gradient Langevin Dynamics) will be used for the numerical optimization of the parameters $\theta$. This method combines a gradient descent step with the

addition of Gaussian noise, ensuring that the distribution converges to the posterior:

$$\theta_0^\lambda := \theta_0, \quad \theta_{n+1}^\lambda = \theta_n^\lambda - \lambda H(\theta_n^\lambda, \mathbf{X}_{n+1}) + \sqrt{\frac{2\lambda}{\beta}}\,\xi_{n+1}, \quad n \in \mathbb{N}_0, \tag{8}$$

where $\lambda > 0$ is the learning rate, $\beta > 0$ is the inverse temperature, which controls the variance of the noise term, and $\xi_{n+1} \sim \mathcal{N}(0, I_d)$.

This optimization scheme allows, on the one hand, convergence toward the minimum of $U(\theta)$ and, on the other, due to the noise added in the update step, more effective exploration of the parameter space, thereby reducing the risk of becoming trapped in local minima.

$$\mathbf{X}_n = (\tau_n, X_{0,n}, Z_n),$$

where:

- $\tau_n \sim \text{Uniform}([0, T])$;
- $X_{0,n} \sim \pi_D$ (for example, $\mathcal{N}(\mu, I_d)$);
- $Z_n \sim \mathcal{N}(0, I_d)$;
- $d$ is the dimension of the parameter vector $\theta \in \mathbb{R}^d$.

After training, we obtain a fixed estimate $\hat{\theta}$ — the weights of the trained neural network. This estimate is then used in the reverse process, which we denote as $Y_t^{\text{aux}}$.

$$dY_t^{\text{aux}} = \left(Y_t^{\text{aux}} + 2\,s(T - t, \hat{\theta}, Y_t^{\text{aux}})\right) dt + \sqrt{2}\,d\bar{B}_t, \quad Y_0^{\text{aux}} \sim \pi_\infty = \mathcal{N}(0, I_M). \tag{9}$$

Thus:

- $\hat{\theta}$ — the weights of the neural network $s$ after training;
- $Y_t^{\text{aux}}$ — the reverse process in which the exact score is replaced by the approximator $s(\cdot, \hat{\theta}, \cdot)$.

This process plays a key role in deriving nonasymptotic bounds in the Wasserstein-2 distance between the true data distribution and the distribution generated by the model. To implement $Y_t^{\text{aux}}$ in practice, the Euler–Maruyama discretization scheme is applied. A uniform time grid is introduced:

$$t_{k+1} = \sum_{j=0}^{k} \gamma_j, \quad t_0 = 0, \quad t_{K+1} = T, \quad \gamma_k = \gamma \in (0, 1),$$

and the discrete process:

$$Y_{k+1}^{\mathrm{EM}} = Y_k^{\mathrm{EM}} + \gamma \left( Y_k^{\mathrm{EM}} + 2\, s(T - t_k, \hat{\theta}, Y_k^{\mathrm{EM}}) \right) + \sqrt{2\gamma}\, \bar{Z}_{k+1}, \quad Y_0^{\mathrm{EM}} \sim \pi_\infty, \tag{10}$$

where $\{\bar{Z}_k\}$ are independent Gaussian vectors with zero mean and unit covariance.

This method constitutes a concrete numerical generation algorithm: starting from Gaussian noise, we iteratively "rewind" the process toward the data distribution using the estimated score.

Thus, the complete workflow is as follows:

1. **Forward process** — diffusion driving the data toward the stationary distribution $\pi_\infty$.
2. **Score matching** — training the approximation $s(t, \theta, x)$ of the exact score $\nabla \log p_t(x)$.
3. **SGLD** — optimization of the parameters $\theta$ via the stochastic gradient.
4. **Reverse process with estimated score** ($Y_t^{\mathrm{aux}}$) — approximate dynamics for data reconstruction.
5. **Euler–Maruyama numerical scheme** ($Y_k^{\mathrm{EM}}$) — discrete implementation of the sample generator.

## 3.2 Convergence in terms of the Wasserstein-2 distance for SGMs

At this stage, it is important to mention that, for further analysis, the similarity between the original distribution and the one obtained after the backward process was evaluated using the Wasserstein-2 distance. This metric provides a rigorous way of quantifying the discrepancy between probability distributions. Formally, it is defined as

$$W_2^2(\mu, \nu) = \inf_{\gamma \in \Pi(\mu,\nu)} \int_{\mathbb{R}^d \times \mathbb{R}^d} \|x - y\|^2 \, d\gamma(x, y),$$

where

- $x, y \in \mathbb{R}^d$ are points in the $d$-dimensional space,
- $\mu$ and $\nu$ are the two probability distributions under comparison,
- $\gamma$ is a *coupling* (or transport plan) between $\mu$ and $\nu$, i.e., a joint distribution on $\mathbb{R}^d \times \mathbb{R}^d$ with marginals $\mu$ and $\nu$,
- $\Pi(\mu, \nu)$ denotes the set of all such couplings.

In the one-dimensional case, the Wasserstein-2 distance admits a closed-form representation in terms of the quantile functions $F_\mu^{-1}$ and $F_\nu^{-1}$:

$$W_2^2(\mu,\nu) = \int_0^1 \left(F_\mu^{-1}(u) - F_\nu^{-1}(u)\right)^2 du,$$

which allows direct computation via quantile transforms.

In the multivariate case, computing the exact Wasserstein-2 distance is computationally demanding. Therefore, in this work we employed the *Sliced Wasserstein distance*, which approximates the Wasserstein-2 metric by projecting high-dimensional distributions onto one-dimensional subspaces:

$$SW_2^2(\mu,\nu) = \int_{S^{d-1}} W_2^2\Big((\langle u,\cdot\rangle)_\#\mu,\ (\langle u,\cdot\rangle)_\#\nu\Big)\, du,$$

where

- $S^{d-1}$ is the unit sphere in $\mathbb{R}^d$,
- $\langle u,\cdot\rangle$ denotes the projection of a vector onto the direction $u \in S^{d-1}$,
- $(\langle u,\cdot\rangle)_\#\mu$ denotes the *pushforward measure* of $\mu$ under this projection.

Formally, if $T : \mathbb{R}^d \to \mathbb{R}$ is defined by $T(x) = \langle u, x\rangle$, then

$$T_\#\mu(B) = \mu(T^{-1}(B)), \quad B \subseteq \mathbb{R} \text{ measurable.}$$

Equivalently, if $X \sim \mu$, then $T_\#\mu$ is the distribution of the random variable $T(X) = \langle u, X\rangle$.

In practice, the integral defining the sliced Wasserstein distance is approximated by averaging over a finite set of random projections.

Table 1: Wasserstein-2 distance results for different distributions

| **Distribution** | **W2 (Sample 1)** | **W2 (Sample 2)** |
|---|---|---|
| Gaussian 1D (Analytical score) | 0.001929 | 0.003003 |
| Gaussian 1D (NN-predicted score) | 0.002928 | 0.002662 |
| Exponential ($\lambda = 1$; NN-predicted score) | 0.4654 | 0.4697 |
| Exponential ($\lambda = 2$; NN-predicted score) | 0.6758 | 0.6501 |
| Mixed Gaussian 1D (NN-predicted score) | 0.201846 | 0.177881 |
| Gaussian 2D (NN-predicted score) | Sliced W2: 0.0599 | |
| Mixed Gaussian 2D (NN-predicted score) | Sliced W2: 0.1647 | |

As can be seen from Table (1), the Wasserstein distance for the one-dimensional Gaussian distribution is very small, both when using the analytical score function and when employing the neural network approximation. This indicates a high degree of agreement between the generated and true samples.

For the exponential distribution, the discrepancies are noticeably larger. These results suggest that learning the score function for distributions with heavy tails and pronounced skewness is a challenging task, which leads to less accurate approximations.

For the one-dimensional Gaussian mixture distribution, the Wasserstein distance takes relatively low values, indicating the model's ability to reproduce the multimodal structure of the distribution, although the accuracy in this case is lower than for the unimodal Gaussian distribution.

For the two-dimensional Gaussian distribution, the similarity between the generated samples and the target distribution $\mathcal{N}(0, I_2)$ was evaluated using the *Sliced Wasserstein distance*, since the exact computation of the Wasserstein-2 metric in higher dimensions is computationally prohibitive. The results turned out to be quite good, demonstrating a high quality of distribution reconstruction.

For the two-dimensional Gaussian mixture distribution, again, the *Sliced Wasserstein distance* was used, for the same reason of computational tractability. The resulting value demonstrates relatively good reconstruction quality in the multivariate setting, confirming that the method is capable of approximating more complex target distributions beyond the Gaussian case.

## 3.3 Algorithm SBTS (Hamdouche & Pham, 2023)

In the Schrödinger Bridge for Time Series problem, the key objective is to construct a stochastic process that is consistent with the initial data distribution while also reproducing a prescribed target distribution at a future time point. Essentially, this amounts to solving an optimal transport problem for probability mass, subject to stochastic diffusion dynamics.

In the discrete-time formulation for time series, SBTS builds a sequence of distributions that "connect" the initial state (data) to the terminal state (target) with minimal deviation from the baseline dynamics. This is achieved through iterative refinement of the drift term, which adjusts the trajectory directions toward the target distribution.

A central component of the algorithm is the approximation of the drift function $a(t, x)$, which describes the deterministic part of the evolution of the process. Since the analytical form of the drift is unknown, it is estimated directly from observed trajectories using a non-parametric method, specifically the Nadaraya-Watson kernel regression.

Suppose we have $M$ trajectories of the process $\{X_{t_1}^{(m)}, \dots, X_{t_N}^{(m)}\}$ obtained from an empirical distribution $\mu$. For a time $t \in [t_i, t_{i+1}]$ and a state $x \in \mathbb{R}^d$, the drift estimate $\hat{a}(t, x; x_i)$ is computed as

$$\hat{a}(t, x; x_i) = \frac{\sum_{m=1}^{M} \left(X_{t_{i+1}}^{(m)} - x\right) F_i\left(t, X_{t_i}^{(m)}, x, X_{t_{i+1}}^{(m)}\right) \prod_{j=1}^{i} K_h\left(x_j - X_{t_j}^{(m)}\right)}{(t_{i+1} - t) \sum_{m=1}^{M} F_i\left(t, X_{t_i}^{(m)}, x, X_{t_{i+1}}^{(m)}\right) \prod_{j=1}^{i} K_h\left(x_j - X_{t_j}^{(m)}\right)}. \tag{11}$$

Here:

- $x_i \in (\mathbb{R}^d)^i$ — the vector of the trajectory history up to time $t_i$;
- $X_{t_j}^{(m)}$ — the value of the $m$-th trajectory at time $t_j$;
- $K_h(\cdot)$ — a kernel function with smoothing parameter $h > 0$, used for local weighting of observations. In this work, the *quartic kernel* is applied:

$$K(x) = (1 - |x|^2)\, \mathbf{1}_{|x| \le 1}, \quad K_h(x) = \frac{1}{h} K\left(\frac{x}{h}\right), \tag{12}$$

  which reduces computational complexity compared to Gaussian kernels.
- The multiplicative factors $\prod_{j=1}^{i} K_h\left(x_j - X_{t_j}^{(m)}\right)$ provide local averaging over trajectories that are close to the current one.

$$F_i(t, x_i, x, x_{i+1}) = \exp\left(-\frac{\|x_{i+1} - x\|^2}{2(t_{i+1} - t)} + \frac{\|x_{i+1} - x_i\|^2}{2(t_{i+1} - t_i)}\right). \tag{13}$$

**Interpretation:**

- The first term in the exponent,

$$-\frac{\|x_{i+1} - x\|^2}{2(t_{i+1} - t)},$$

  penalizes the deviation of the intermediate point $x$ from the end point of the segment $x_{i+1}$, with the penalty scale depending on the remaining time $(t_{i+1} - t)$.
- The second term,

$$+\frac{\|x_{i+1} - x_i\|^2}{2(t_{i+1} - t_i)},$$

  normalizes the weight with respect to the total variance over the interval $[t_i, t_{i+1}]$.

- The final weight $F_i$ is larger when the intermediate point $x$ is closer to the trajectory passing through $x_i \to x_{i+1}$, in accordance with the Gaussian transition model.

$F_i$ is, in fact, an approximation of the ratio of the process's transition densities between time slices, and thus it accurately reflects the conditional expectations underlying the drift construction in the Schrödinger Bridge. Owing to this weighting, the method accounts for the geometry of trajectories and the temporal structure of the data, rather than merely averaging over the nearest points.

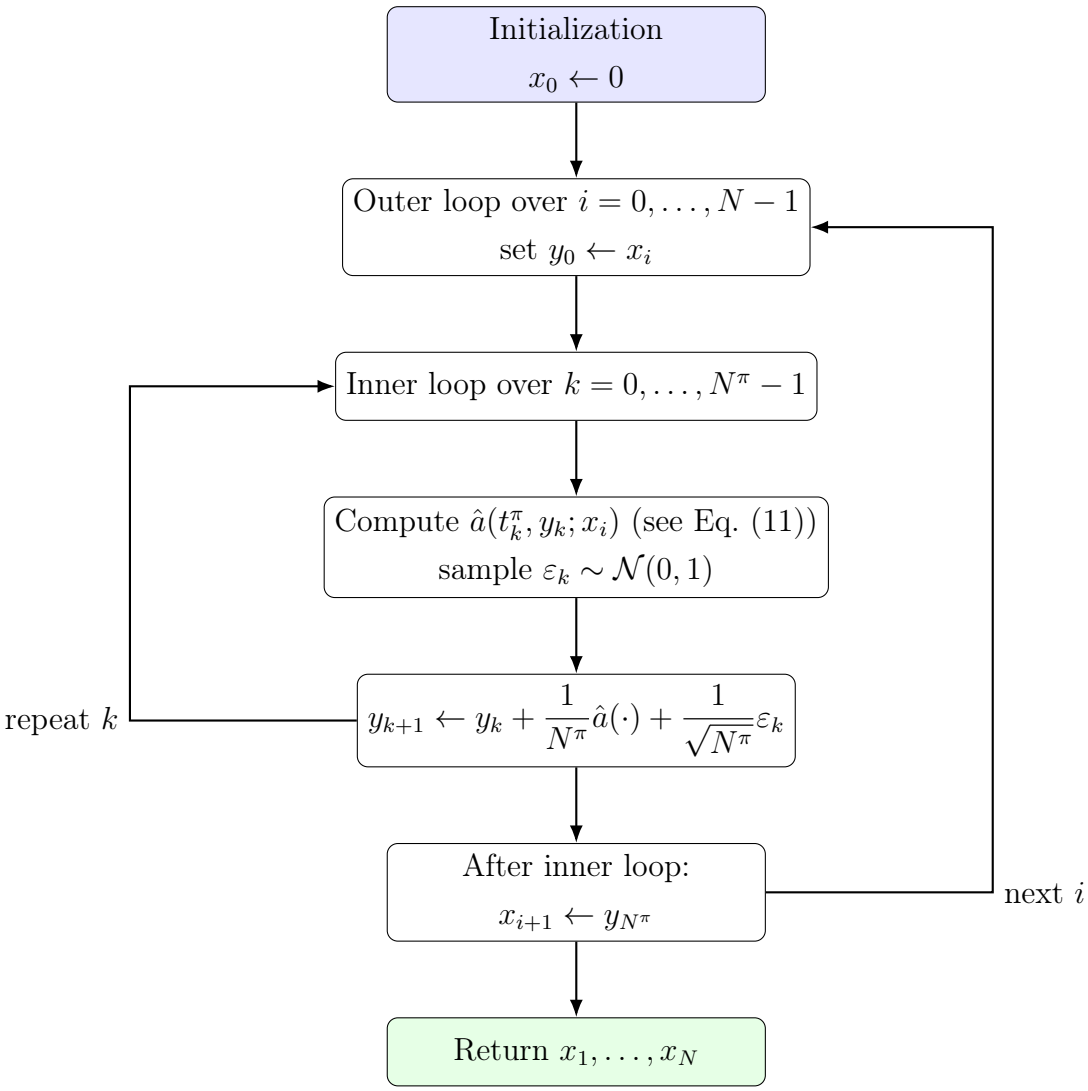



**Figure 1:** Execution flow of the SBTS algorithm.

The intuition is that the SBTS method constructs a stochastic process linking the initial and terminal distributions in such a way that the trajectories align as closely as possible with the stochastic properties of the original data. Rather than approximating the drift with a parametric model (e.g., a neural network), as in diffusion models, SBTS estimates it directly from the training samples using nonparametric methods.

The key idea is a local approximation of the drift based on kernel smoothing and a weighting function that incorporates both spatial and temporal proximity between points. As a result, the estimated drift $\hat{a}(t, x, x_i)$ serves as the principal mechanism guiding the stochastic process toward the target distribution, while the noise term represents random fluctuations around this direction.

Trajectory simulation is performed step-by-step: at each iteration, the state is updated using the estimated drift and Gaussian noise, enabling the generation of new samples that preserve the structure of the training data without exactly replicating them.

# 4 Results

## 4.1 One-Dimensional Gaussian Process: Analytical Score and Estimation of $\theta$ (Appendix 1)

To verify the correctness of both the implementation and the theoretical statements outlined above, we consider an elementary test problem in which the initial data are normally distributed:

$$X_0 \sim \mathcal{N}(\mu, I_p),$$

where the vector of true parameters is set as $\mu = (1, -2)$. In the present study, however, we restrict ourselves to the one-dimensional case. The system's evolution, i.e., the *annealing*, is described by the Ornstein–Uhlenbeck process (see Equation (2)).

It should be noted that in Bruno et al., 2025, the sampling of such a process and, most importantly, the density $p_t(x)$ are discussed. In this case, the density is Gaussian, distributed as

$$\mathcal{N}\big(m_t\mu,\, 1\big).$$

The log-gradient is thus computed as

$$\nabla \log p_t(x) = -x + m_t\mu. \tag{14}$$

In the score-function implementation, we employ the form

$$s^*(t, x_t) = -x_t + m_t\theta, \tag{15}$$

where the parameter $\theta$ serves as an estimator of the true value $\mu$.

The training procedure is based on the method of *stochastic gradient descent*, although

in this case it reduces to a simplified elementary form. Specifically, we minimize

$$H(\theta, x) = 2\sigma_t^2 \sum_{i=1}^{d} \big(\sigma_t^{-1} z^{(i)} + (-m_t x_0^{(i)} - \sigma_t z^{(i)} + m_t \theta)\big) m_t e_i$$

which simplifies to

$$= 2\sigma_t^2 m_t \big(\sigma_t^{-1} z - m_t x_0 - \sigma_t z + m_t \theta\big).$$

The parameter updates are carried out according to formula (8).

The key elements of the experiment include the choice of parameters and the analysis of their influence on convergence. The adopted configuration was as follows:

- The true parameter value is $\mu = 1$ in the first case and $\mu = -2$ in the second.
- The initial value $\theta = 0$ in both cases, which makes it possible to clearly demonstrate convergence.
- Step size: $\eta = 10^{-3}$. A small step ensures stability and smooth trajectories of $\theta$.
- Noise variance: $\beta = 10^{-8}$. A larger value increases the amplitude of SGLD noise, confirming that the method approaches gradient descent.
- Number of iterations: 30000, which suffices for $\theta$ to reach the stationary regime and stabilize around the true value.

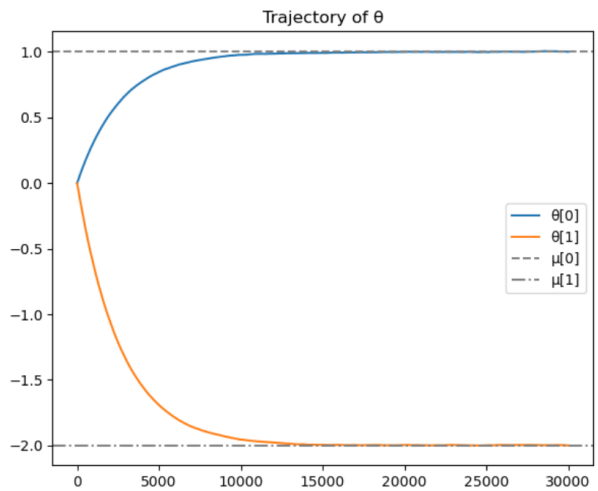



**Figure 2:** Trajectory of the parameter vector $\theta$ during training.

The results (Figure 2) demonstrate that the convergence to the true value is rapid. Hence, this test facilitated the understanding of the system while simultaneously ensuring that the theory of the Ornstein–Uhlenbeck process was implemented correctly, with $\theta$ approximating $\mu$ asymptotically.

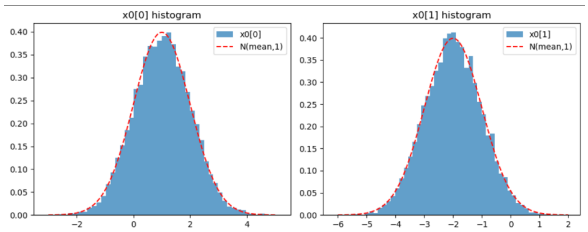



**Figure 3:** Reverse process obtained via the Euler–Maruyama scheme.

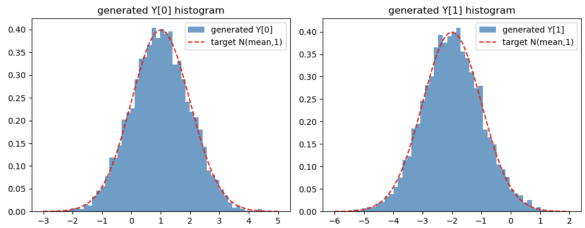



**Figure 4:** Histogram of initial samples with target distribution.

After training, we evaluated the accuracy of data reconstruction through the reverse process (Figure 3). For this purpose, the Euler–Maruyama discretization scheme (Equation (10)) was employed, making use of the analytical score function. The histograms of the generated samples exhibit an almost perfect agreement with the target distribution (Figure 4), thereby confirming the validity of the reconstruction procedure and demonstrating that the reverse dynamics faithfully reproduce the underlying data distribution.

The next stage of the study involved applying formula (7) to compute the stochastic gradient and subsequently estimate the parameter $\theta$. The purpose of this step was to verify the correctness of the performed calculations: at this stage, such validation is feasible, since the results allow for a direct comparison with the reference values.

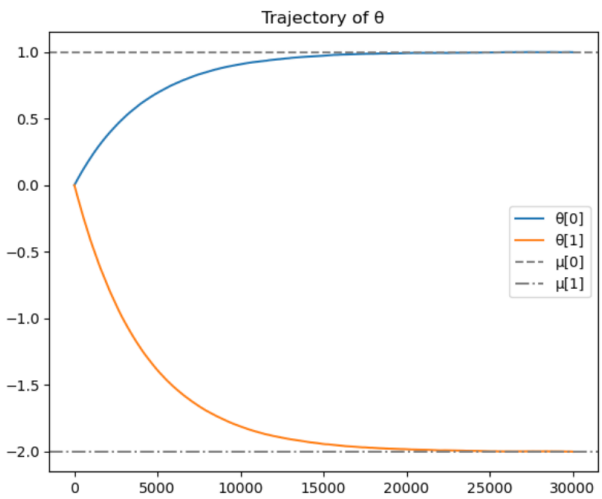


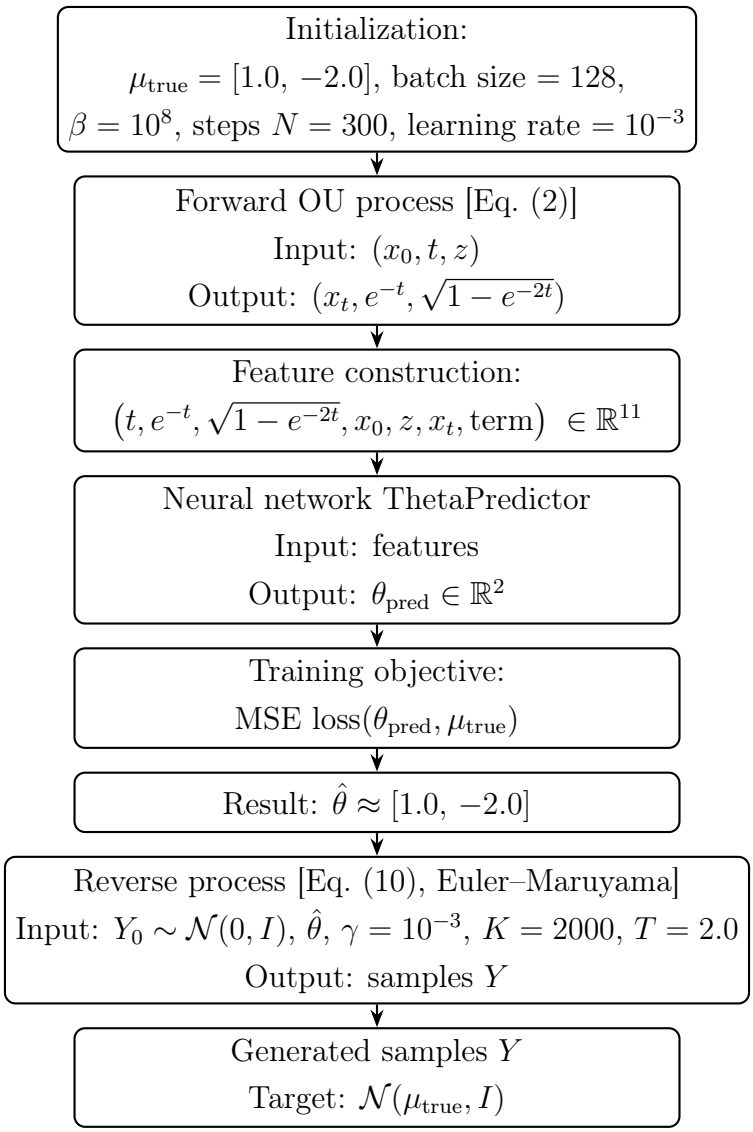

**Figure 5:** Trajectory of the parameter vector $\theta$ during training using formula (7).

Figure 5 presents the convergence dynamics of the $\theta$ trajectories, which exhibit a clear resemblance to the process illustrated in Figure 2. This agreement confirms the correctness of the calculations performed and serves as an indicator of readiness for further problem generalization. In particular, subsequent experiments are intended to address the case in which the score function is unknown. In such a setting, the computation of the score-function gradient becomes significantly more demanding: these derivations can no longer be carried out straightforwardly "by hand" and instead require the use of numeri-

cal methods. Consequently, direct verification of computational accuracy becomes more challenging, making it reasonable to ensure the correctness of the procedure in advance, at the stage when the analytical form of the score is still accessible.

## 4.2 Estimating $\theta$ with a neural network (Appendix 1)

In contrast to the previous case, where $\theta$ was obtained using SGLD, here the analytical form of the OU score function was preserved, while the estimation of the parameter $\theta$ was delegated to a small neural network. This setup models a situation in which the structure of the score is known theoretically, but its parameters must be further estimated from the data.




The neural network is designed with the following architecture:

$$\text{Input}(11) \longrightarrow \text{Linear}(64) \longrightarrow \text{ReLU} \longrightarrow \text{Linear}(64) \longrightarrow \text{ReLU} \longrightarrow \text{Linear}(2).$$

Since in formula (15) all the required quantities are known at each moment in time, it becomes possible to directly compute $\nabla \log p_t(x)$ using relation (14). Subsequently, for training the model one can employ a standard loss function, namely the mean squared error (MSE) between the predicted parameters $\Theta_{\text{pred}}$ and the true value $\mu$.

$$\mathcal{L}(\theta) = \|\theta_{\text{pred}} - \mu\|_2^2 \,,$$

This approach provides explicit control over the optimization process and ensures the correctness of aligning the model's predictions with the reference parameters.

The configuration of the training parameters remained unchanged, except for two modifications: the number of steps was increased to 300, and an additional parameter—the batch size, set to 128—was introduced.

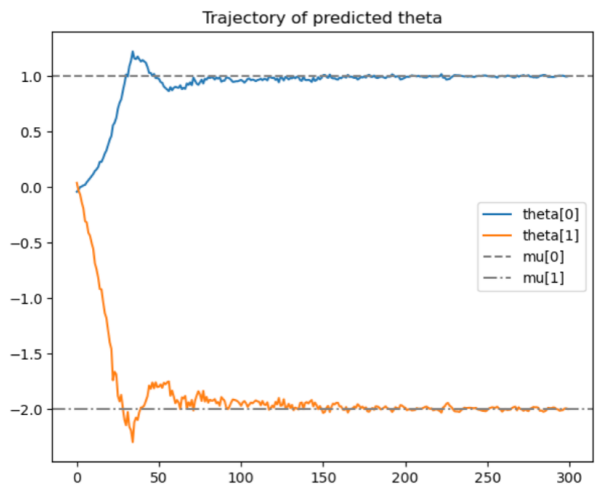



**Figure 6:** Trajectory of the parameter vector $\theta$ using NN.

Figure 6 presents the results of training the neural network aimed at recovering the parameter $\theta$. It can be observed that the trajectories of the predicted components of $\theta$ during optimization converge rather quickly to their true values. At the initial stage, oscillations are present, caused by the stochasticity of mini-batch formation and the inherent noise in the data. However, after approximately fifty iterations the process stabilizes, and by around the hundredth iteration the predicted values almost perfectly coincide with the target ones.

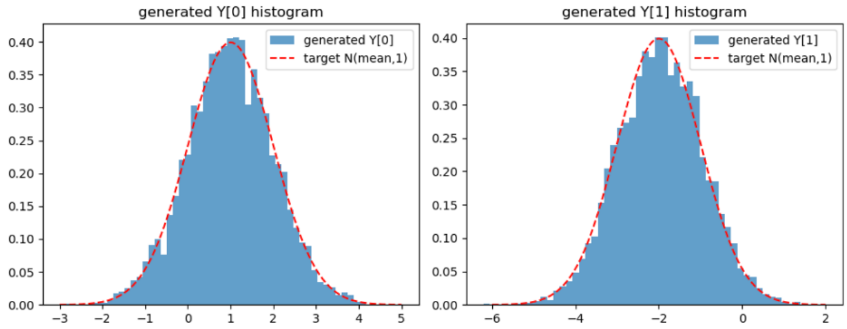



**Figure 7:** Histogram of samples with $\theta$ predicted by NN.

Figure 7 shows the outcome of the backward process. As can be seen, the generated data nearly perfectly reproduces the original distribution depicted in Figure 3, which confirms the validity of the implemented approach and the high accuracy of recovery.

## 4.3 Two-dimensional Gaussian distribution (Appendix 2)

To move toward more complex scenarios, it was decided to investigate a two-dimensional distribution with parameters $\mu = (1, -2)$ and $\Sigma = I$. In contrast to the one-dimensional case, here the score function was approximated using a neural network. As the model, an extremely simple architecture was chosen—a two-layer neural network whose structure corresponds to the classical formulation presented in (Mei et al., 2018).

The two-layer neural network with $N$ hidden units can be written in the mean-field form as

$$\hat{y}(x;\theta) = \frac{1}{N}\sum_{i=1}^{N} \sigma_*(x;\theta_i), \qquad \sigma_*(x;\theta_i) = a_i\,\sigma\big(\langle w_i, x\rangle + b_i\big), \tag{16}$$

where $\theta_i = (a_i, w_i, b_i)$ denotes the parameters of the $i$-th neuron, $\sigma(\cdot)$ is the activation function, $w_i \in \mathbb{R}^d$ is the weight vector, $b_i \in \mathbb{R}$ is the bias term, and $a_i \in \mathbb{R}$ is the output weight.

This model was taken as the basis for constructing an approximation of the score function, with the neural network now serving as a predictor of the parameter $\theta$. Within this approach, $\theta$ is interpreted as the set of weights of the neural model.

In subsequent experiments, the overall training procedure will remain fixed; only certain hyperparameters will be varied. Specifically, in this model the batch size is set to 16, the learning rate to $10^{-3}$, and the number of steps to 10,000.

At the next stage, the question arose as to which characteristics should be monitored in order to assess the correctness of the model training. As such an indicator, the norm of the stochastic gradient $H$ was chosen, as it provides a clear measure of the optimization dynamics.

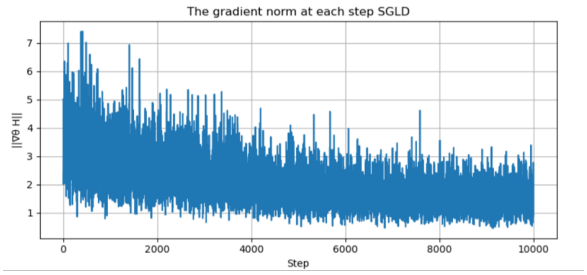



**Figure 8:** Evolution of the stochastic gradient norm $\|H(\theta, x)\|$ at each training step during SGLD optimization.

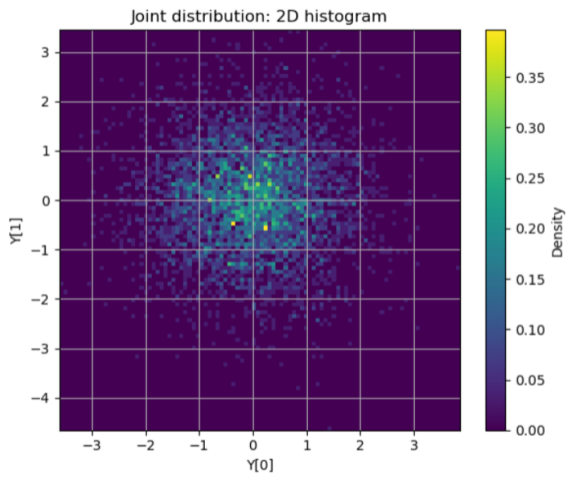



**Figure 9:** Distribution of the generated sample, visualized as a two-dimensional histogram.

Figure 8 illustrates the dynamics of the gradient norm throughout all training steps. A gradual decrease followed by stabilization can be observed, which confirms the correct convergence of the method based on the two-layer score-function architecture.

The results of the backward process, presented in Figure 9 specifically, data generation according to formula (10) demonstrate that the obtained samples closely follow the normal distribution in the two-dimensional space. Thus, this experiment shows that even in such a setting it is possible to successfully reproduce the true distribution.

## 4.4 One-dimensional exponential distribution (Appendix 3, 4)

At the next stage, the study moves from the symmetric Gaussian distribution to an asymmetric and one-sided law—namely, the exponential distribution. The main objective is to test the robustness of the score-function training procedure as well as the correctness of the reverse dynamics implementation.

As input data, we consider vectors

$$X_0 = \left(X_0^{(1)}, X_0^{(2)}\right), \quad X_0^{(i)} \sim \mathrm{Exp}(\lambda),$$

where the components are independent. Two experiments are planned with parameters $\lambda = 1$ and $\lambda = 2$.

The forward process remains unchanged—it is described by the Ornstein–Uhlenbeck diffusion, which ensures controlled degradation of the signal into a Gaussian state. However, due to the specifics of the exponential distribution, it was necessary to introduce certain modifications into the model in order to constrain the trajectories $X_t$ to the domain of

nonnegative values.

In the case of the one-dimensional exponential distribution, the approximation of the score function is performed using a more sophisticated neural network architecture: a four-layer perceptron with ReLU activation functions. Training is still carried out by means of stochastic gradient Langevin dynamics (SGLD, see formula (7)).

The choice of this scheme is motivated by the properties of the exponential distribution. Being one-sided and exhibiting pronounced sharpness near the boundary $x = 0$, it induces highly nonuniform contributions to the functional $H$ across both time and space. This effect is particularly evident at the early stages of evolution, when the variance of the OU noise remains small and the influence of the characteristics of the original distribution is most significant.

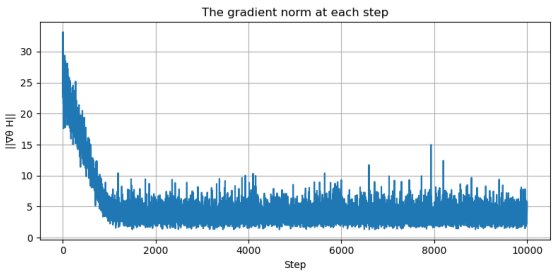



**Figure 10:** Norm of the stochastic gradient $\|H(\theta, x)\|$ during training on the Exp(1) distribution.

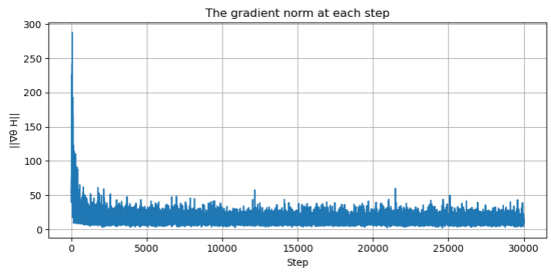



**Figure 11:** Norm of the stochastic gradient $\|H(\theta, x)\|$ during training on the Exp(2) distribution.

During the experiments, a characteristic behavior of the gradient norm was observed: its value is initially large, but as training progresses it rapidly decreases and subsequently stabilizes at a comparatively low level.

For the case $\lambda = 1$ (see Figure 10), the stationary values of the gradient norm lie in the range from 5 to 8. For $\lambda = 2$ (see Figure 11), the initial value is considerably higher, while the stabilized level is less smooth. This effect is explained by the steeper logarithmic gradient of the density near zero, which leads to increased variability in the estimates.

Thus, the initial peak may be interpreted as the phase during which the coarse parameters of the neural network are aligned with the statistics of the trajectories of the Ornstein–Uhlenbeck forward process. The subsequent monotonic decline reflects the proper training of the network to compensate for the deterministic drift of the OU process. Finally, the fluctuations in the late stage of convergence are caused by individual minibatches corresponding to small values of $\varepsilon$; in precisely these cases the gradient norm temporarily increases.

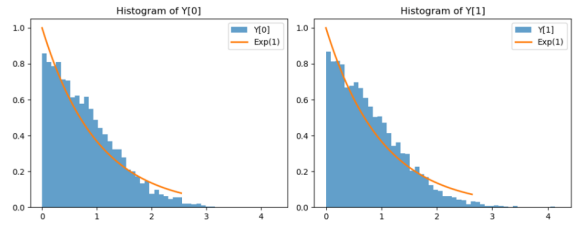



**Figure 12:** Histograms of the generated samples overlaid with the theoretical exponential density Exp(1).

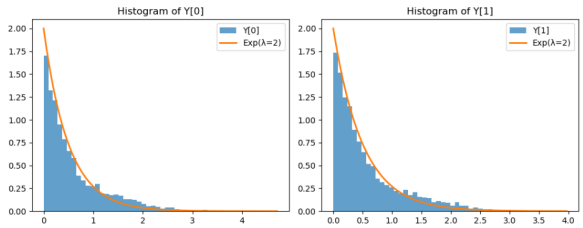



**Figure 13:** Histograms of the generated samples overlaid with the theoretical exponential density Exp(2).

After training was completed, the weights $\hat{\theta}$ were fixed, and the reverse process was launched according to formula (10). As shown in the histograms in Figure 12 ($\lambda = 1$) and Figure 13 ($\lambda = 2$), the generated distributions almost perfectly coincide with the theoretical densities. Minor discrepancies are explained by the discretization error of the Euler–Maruyama scheme. Nevertheless, this effect is statistically negligible and does not distort either the tails or the central part of the distribution.

A comparison of the two series of experiments highlights the key properties of the method:

**Model identifiability.** Variation of the parameter $\lambda$ directly affects the shape of the score function. At the same time, the neural approximation, without explicit knowledge of the value of $\lambda$, after training reproduces the required exponential dependence, which indicates correct learning of the distribution's structure.

**Optimization "stiffness."** With increasing $\lambda$, the logarithmic density becomes steeper near $x = 0$. This leads to a sharp increase in the target drift in the reverse stochastic differential equation, making the optimization procedure more sensitive and variable.

## 4.5 One-dimensional Mixture of Gaussian (Appendix 5)

We then proceeded to examine a one-dimensional mixture of two Gaussian distributions with equal weights:

$$\pi_D(x) = \tfrac{1}{2}\mathcal{N}(x \mid \mu_1, \sigma^2) \; + \; \tfrac{1}{2}\mathcal{N}(x \mid \mu_2, \sigma^2), \quad x \in \mathbb{R},$$

where the parameters are chosen as follows:

$\mu_1 = 0,\ \mu_2 = 3$ — means of the components, $\sigma^2 = 1$ — common variance, $\pi = 0.5$ — equal mixture weights.

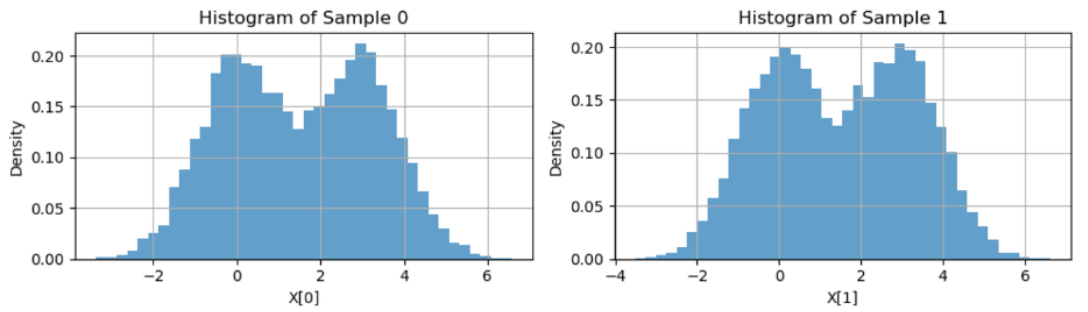



**Figure 14:** Histograms of the two components of the generated samples from the 1D Gaussian mixture.

Thus, the target distribution is characterized by the presence of two distinct modes, which makes the problem substantially more challenging compared to the previously considered unimodal cases. Figure 14 presents histograms of empirical samples along the two coordinates of the random vector, clearly demonstrating the pronounced multimodality.

The score-function approximation is again implemented using a four-layer neural network with ReLU activations. Both the training procedure and the architecture remain identical to the previous experiments. In this way, we assess the robustness and correctness of the algorithm.

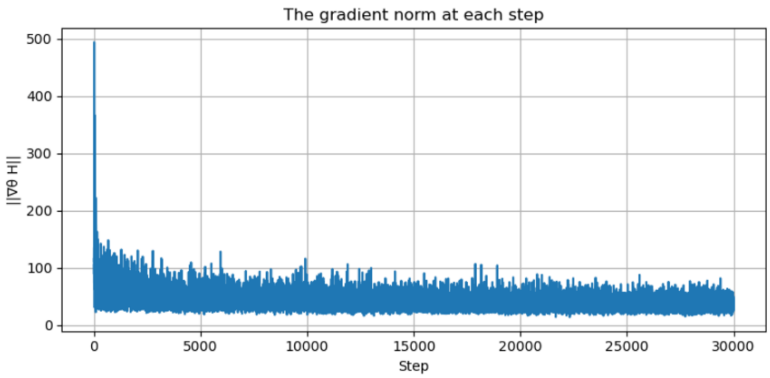



**Figure 15:** Gradient norm at each training step during score model learning on the one-dimensional Gaussian Mixture Model.

Figure 15 illustrates the gradient norm $\|\nabla H\|$ as a function that depends on the iteration number. A pronounced peak is observed in the initial phase, corresponding to the alignment of the randomly initialized network parameters with the data statistics, followed by a rapid decay and convergence to a stationary level. Local fluctuations are explained by minibatches at small time values $t$.

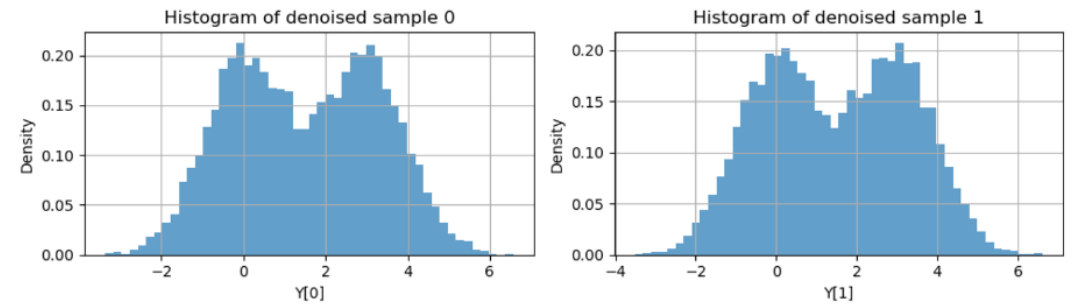



**Figure 16:** Histograms of the denoised samples obtained through the reverse diffusion process applied to a one-dimensional Gaussian Mixture Model.

After training, the parameters $\hat{\theta}$ were fixed by selecting the best-performing values, after which the backward process was executed. Figure 16 presents histograms of the samples obtained after denoising. A comparison with the original distributions demonstrates complete agreement: both the central region and the tails of the distributions remain consistent. Minor discrepancies are attributable to the discretization error of the numerical scheme and are statistically insignificant.

## 4.6 Two-dimensional Mixture of Gaussian (Appendix 6)

For the next stage of increasing complexity, we proceeded to a two-dimensional Gaussian mixture distribution with equal weights. The target density was defined as follows:

$$\pi_D(x) = \tfrac{1}{2}\mathcal{N}(x \mid \mu_1, \Sigma) + \tfrac{1}{2}\mathcal{N}(x \mid \mu_2, \Sigma), \quad x \in \mathbb{R}^2,$$

where $\mu_1, \mu_2 \in \mathbb{R}^2$ are the mean vectors, $\Sigma$ is the common covariance matrix, and the mixture weights are $\pi_1 = \pi_2 = 0.5$.

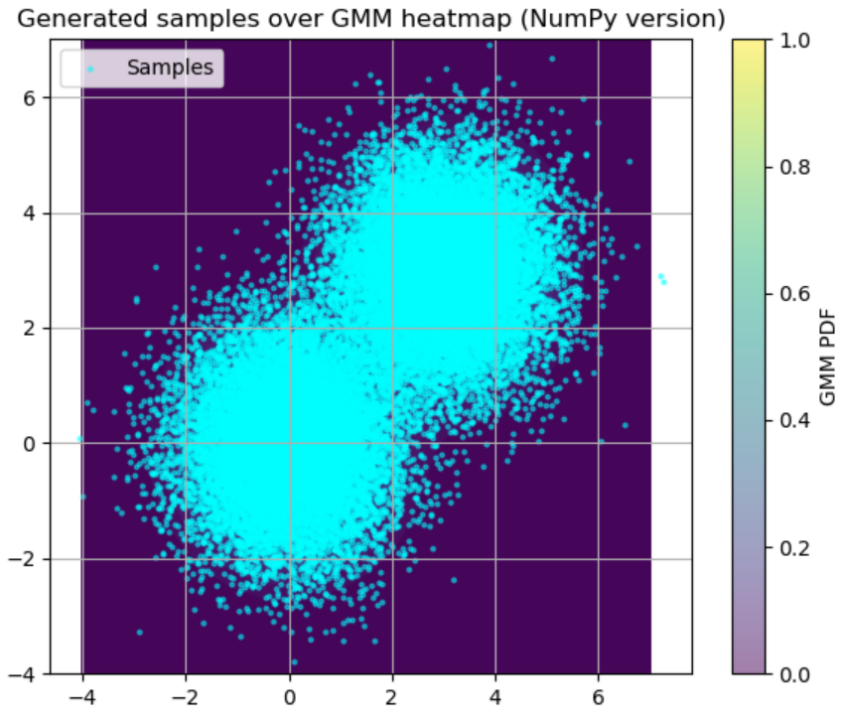



**Figure 17:** Generated samples (cyan) overlaid on the heatmap of the target two-dimensional Gaussian Mixture Model.

Figure 17 shows a sample distribution from this mixture overlaid on a heatmap of the density. It can be seen that the data form two well-separated clusters corresponding to the modes of the Gaussians.

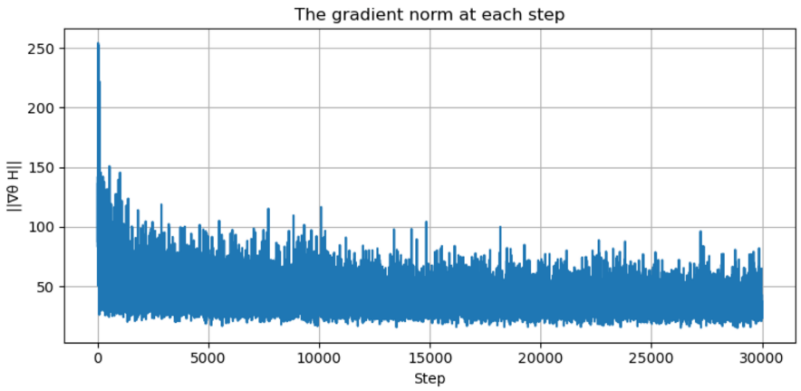



**Figure 18:** Gradient norm at each training step for the score model on a two-dimensional Gaussian Mixture Model.

The score function was again approximated using a four-layer neural network with ReLU activation functions. Figure 18 displays the behavior of the gradient norm. As in the one-dimensional case, a pronounced initial peak is observed, followed by a sharp decline and convergence to a stationary level accompanied by fluctuations. This gradient dynamics confirms the model's stable adaptation to the forward process task.

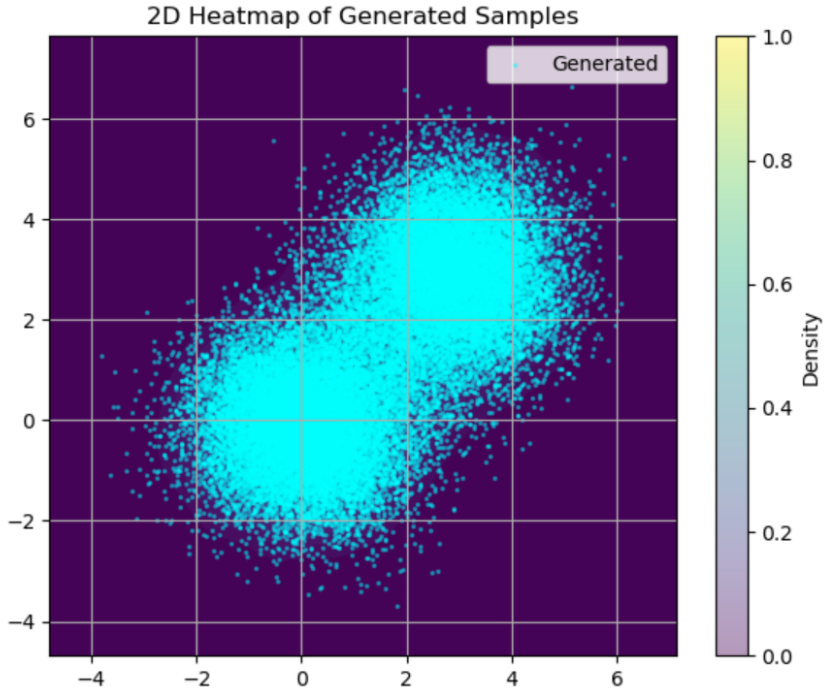



**Figure 19:** Two-dimensional heatmap of samples generated by the reverse diffusion process.

After training, the parameter $\hat{\theta}$ was fixed, and the backward process was executed using the Euler–Maruyama scheme (10). Figure 19 presents the generated samples superimposed on the heatmap of the empirical density. A comparison with Figure 17 shows an almost perfect match. Ultimately, the transition to the two-dimensional Gaussian mixture distribution demonstrates the method's ability to handle more complex multimodal cases, thereby confirming its universality.

## 4.7 Portfolio generation using SGMs (Appendix 7, 8, 9, 10, 11)

All preliminary experiments were essential to ensure the correctness of the code implementation and to gain a deeper understanding of the theoretical foundations of the method. After these validations, the stochastic generative model (SGM) was adapted to a more applied task — generating return distributions of a stock portfolio for the purpose of analyzing potential profitability.

For the experiment, five of the largest technology companies were selected: Apple, Amazon, Microsoft, Google, and Netflix. Training was conducted on historical data spanning the period from January 1, 2010, to December 31, 2019. Subsequently, the time window was shifted forward in six-month increments while maintaining a comparable sample length. Since the data were available up to May 31, 2022, it was possible to construct and test the model on five distinct time intervals.

The idea of the investment experiment was as follows: to invest a notional USD 100,000

and hold the positions for two months without exiting the market. The choice of the baseline period (2010–2019) was motivated by its relative stability, as no global shocks occurred in the market during this time. However, by late 2019 the COVID-19 pandemic began, triggering sharp fluctuations and declines in the stock prices of many companies. Shifting the time intervals thus made it possible to test the robustness of the method under more turbulent market conditions.

It is important to emphasize that the study focuses specifically on the daily returns of large companies. As a preprocessing step, log-returns are computed from prices $P_t$ as $\log(P_t/P_{t-1})$, after which the time series are cleaned of missing values and appropriately scaled.

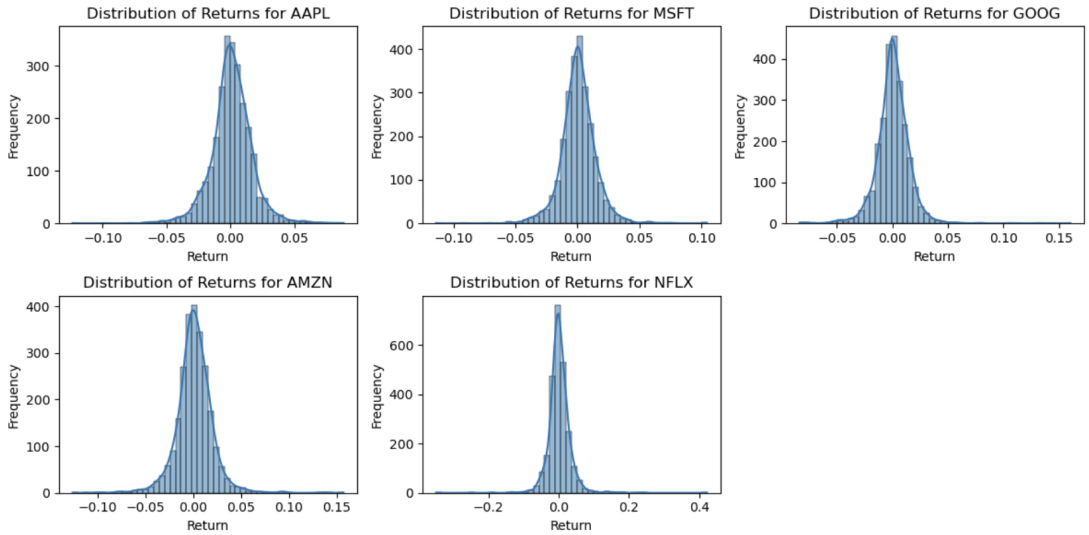



**Figure 20:** Distribution of returns.

The empirical histograms (Figure 20) exhibit properties typical of financial data: sharp peaks, asymmetry, and heavy tails. These features are important, as they put additional strain on the reverse process and increase the accuracy requirements for score-function approximation. The overall model remains unchanged, with the same methodology as described in Section 3.1 being employed.

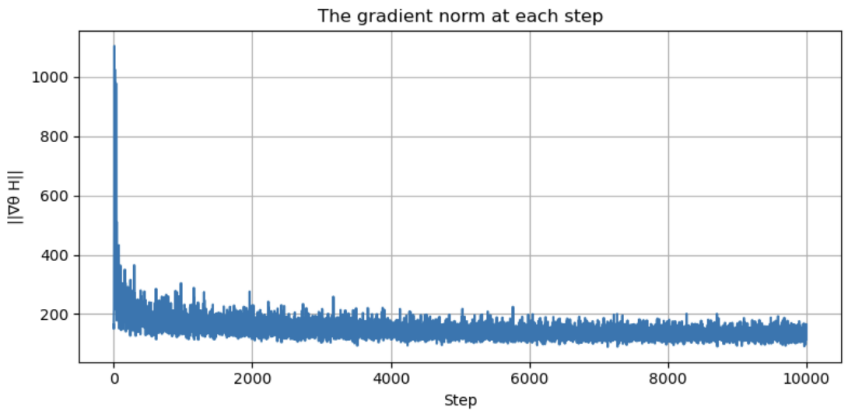



**Figure 21:** Gradient norm at each training step for financial data.

Figure 21 illustrates the evolution of the norm $\|\nabla H\|$ across iterations. Once again, a high initial peak is observed, followed by a decline and stabilization at a stationary level. As can be seen, the dynamics indicate proper optimization behavior; however, the convergence level remains comparatively high, which indirectly points to the difficulty of approximating distributions with heavy tails.

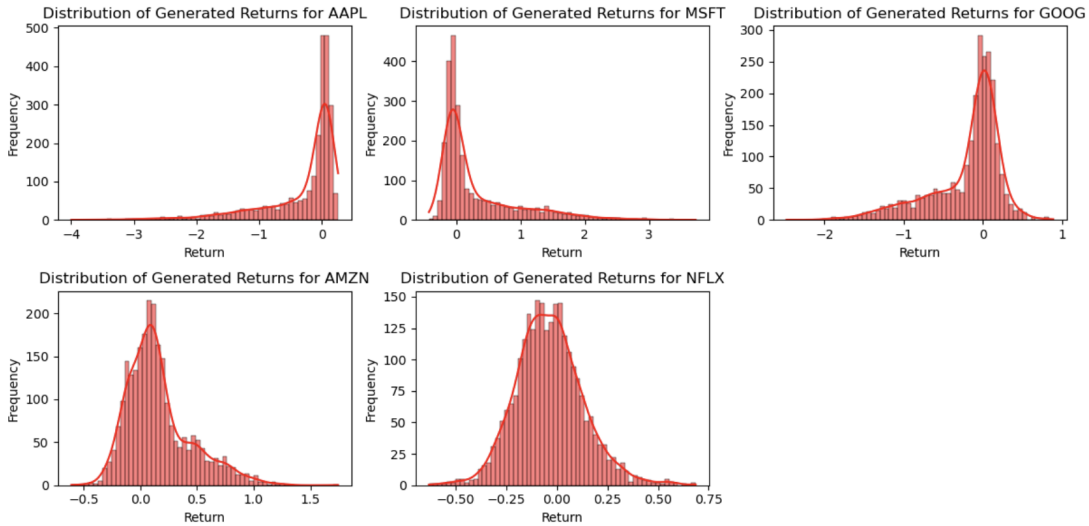



**Figure 22:** Generated distribution of returns.

The results of generating the return distributions are presented in Figure 22 for the same set of tickers. The central parts of the distributions (around the median) are reproduced satisfactorily: both the mode and the overall peak width are close to the empirical ones. However, for several tickers noticeable shifts in the mean or variance are observed. The tails are in some cases overestimated or underestimated (for certain assets the left tail appears excessively heavy, for others the right tail, while in some instances the network smooths out extreme outliers). Taken together, this indicates that the current network configuration and noise regime in SGLD already capture the overall behavior of the distributions, but the calibration with respect to extreme probabilities remains incomplete.

From a practical standpoint, this outcome is to be expected, since financial time series exhibit heteroskedasticity and volatility clustering, which are poorly aligned with a stationary OU diffusion and therefore require more refined tuning.

Accordingly, hyperparameter selection (learning rate, noise injection level in SGLD, mini-batch size) was performed and yielded a workable regime, yet the model remains sensitive. In other words, there is still room for adjustment to improve the approximation of the tails: the obtained parameters are functional but call for further investigation. As a potential direction, one could consider extending the neural network architecture for the score function, or alternatively, replacing the OU process with a time-dependent diffusion.

The optimal asset weights were determined using the criterion of maximizing the Sharpe Ratio (SR):

$$\max_{w} SR(w) = \frac{\mathbb{E}\big[R_p(w)\big]}{\sigma_p(w)},$$

where $R_p(w)$ denotes the return of the portfolio with weights $w$, and $\sigma_p(w)$ is its standard deviation.

This approach offers several advantages: it accounts simultaneously for both the expected return and the portfolio risk; unlike the classical Markowitz mean–variance optimization, the Sharpe Ratio is directly interpretable as a measure of the efficiency of risk utilization.

Table 2: Portfolio generation results based on SGMs (initial investment: \$100,000, horizon: 2 months).

| Period | AAPL | MSFT | GOOG | AMZN | NFLX | Final Value | Return |
|---|---|---|---|---|---|---|---|
| 2020-01-01 – 2020-03-01 | 0.1706 | 0.2405 | 0.2532 | 0.2138 | 0.1220 | \$149,350.10 | +49.35% |
| 2020-06-01 – 2020-09-01 | 0.1721 | 0.2769 | 0.2911 | 0.2598 | 0.0000 | \$148,165.83 | +48.17% |
| 2021-01-01 – 2021-03-01 | 0.2157 | 0.2949 | 0.2802 | 0.1936 | 0.0156 | \$141,283.34 | +41.28% |
| 2021-06-01 – 2021-09-01 | 0.2477 | 0.2458 | 0.2614 | 0.2195 | 0.0256 | \$146,243.15 | +46.24% |
| 2022-01-01 – 2022-03-01 | 0.2569 | 0.2670 | 0.1842 | 0.1851 | 0.1068 | \$70,270.62 | –29.73% |

The obtained results demonstrate the high effectiveness of SGMs: in four out of five cases the model generated portfolios with substantial two-month returns. However, at the beginning of 2022 the quality of the results declined sharply, as the model failed to correctly capture the change in volatility and recorded a loss of nearly 30 percents.

### 4.8 Results Schrödinger Bridge Algorithm 3.1 (Appendix 12, 13, 14, 15, 16)

In contrast to diffusion-based models (SGMs), which are designed to recover stochastic dynamics through neural network approximation of the score function, the SBTS algorithm constructs trajectories according to the principle of optimal stochastic transport between the empirical data distribution and the target distribution. When applied to time series, this implies that the model adjusts trajectories in such a way as to reproduce the observed statistical characteristics of returns while deviating as little as possible from the baseline diffusion dynamics.

Table 3: Portfolio generation results based on SBTS (initial investment: $100,000, horizon: 2 months).

| Period | AAPL | MSFT | GOOG | AMZN | NFLX | Final Value | Return |
|---|---|---|---|---|---|---|---|
| 2020-01-01 – 2020-03-01 | 0.3338 | 0.3356 | 0.3607 | -0.2754 | 0.2454 | $139,840.53 | +39.84% |
| 2020-06-01 – 2020-09-01 | 0.4623 | 0.0213 | 0.3235 | -0.1041 | 0.2969 | $150,458.23 | +50.46% |
| 2021-01-01 – 2021-03-01 | -0.4345 | -0.1726 | 0.4675 | -0.4702 | 1.6098 | $139,804.27 | +39.80% |
| 2021-06-01 – 2021-09-01 | 2.1663 | 0.4231 | -2.0192 | -1.4758 | 1.9056 | $96,126.57 | -3.87% |
| 2022-01-01 – 2022-03-01 | 0.4662 | -0.3727 | 0.3580 | 0.7877 | -0.2393 | $64,986.37 | -35.01% |

The interpretation of the obtained results can be summarized as follows:

- **High returns during periods of market stability and recovery** (e.g., in 2020) are explained by the ability of SBTS to adequately reproduce the central part of the distribution and to capture short-term temporal patterns in the data.
- **Instability of portfolio weights**, manifested in negative or excessively inflated coefficients, is attributable to the use of local kernel-based approximation, which is highly sensitive to noise and outliers.
- **Reduced adaptability under crisis dynamics** (2021–2022) stems from the model's underlying assumption of quasi-stationarity, whereas real financial time series are characterized by pronounced heteroscedasticity and volatility clustering.

# Conclusion

In this study, we conducted a comprehensive investigation of stochastic generative models (SGMs) based on score-function approximation and the reverse dynamics of diffusion processes.

Our experiments were carried out not only on simple distributions but also in two-dimensional multimodal settings. The results show that different types of neural networks can accurately capture and recreate the structure of target distributions. We also used this method to solve the problem of creating a portfolio with the goal of maximizing the Sharpe ratio. We compared the results we got using SGMs (Table 2) to those from another method based on the Schrödinger Bridge (Table 3).

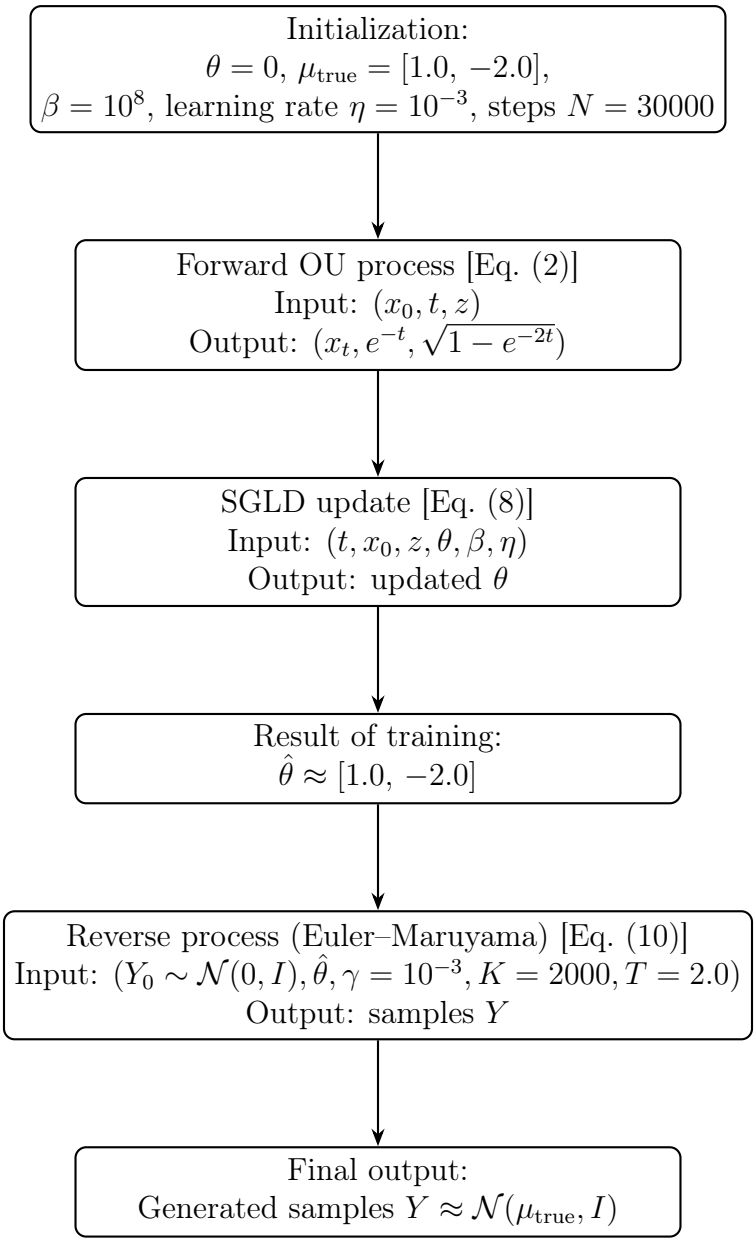
The comparative analysis showed that, within the scope of our experiments, the SGM-based approach provided superior portfolio generation in 4 out of 5 cases relative to SBTS. This empirical evidence indicates that, in our setting, SGMs are more effective and robust than SBTS, thereby confirming their versatility and adaptability. Strong performance both in synthetic experiments and in the real-world financial application highlights the significant potential of this methodology and motivates further research aimed at improving and extending these results.

# Appendix: Code Availability

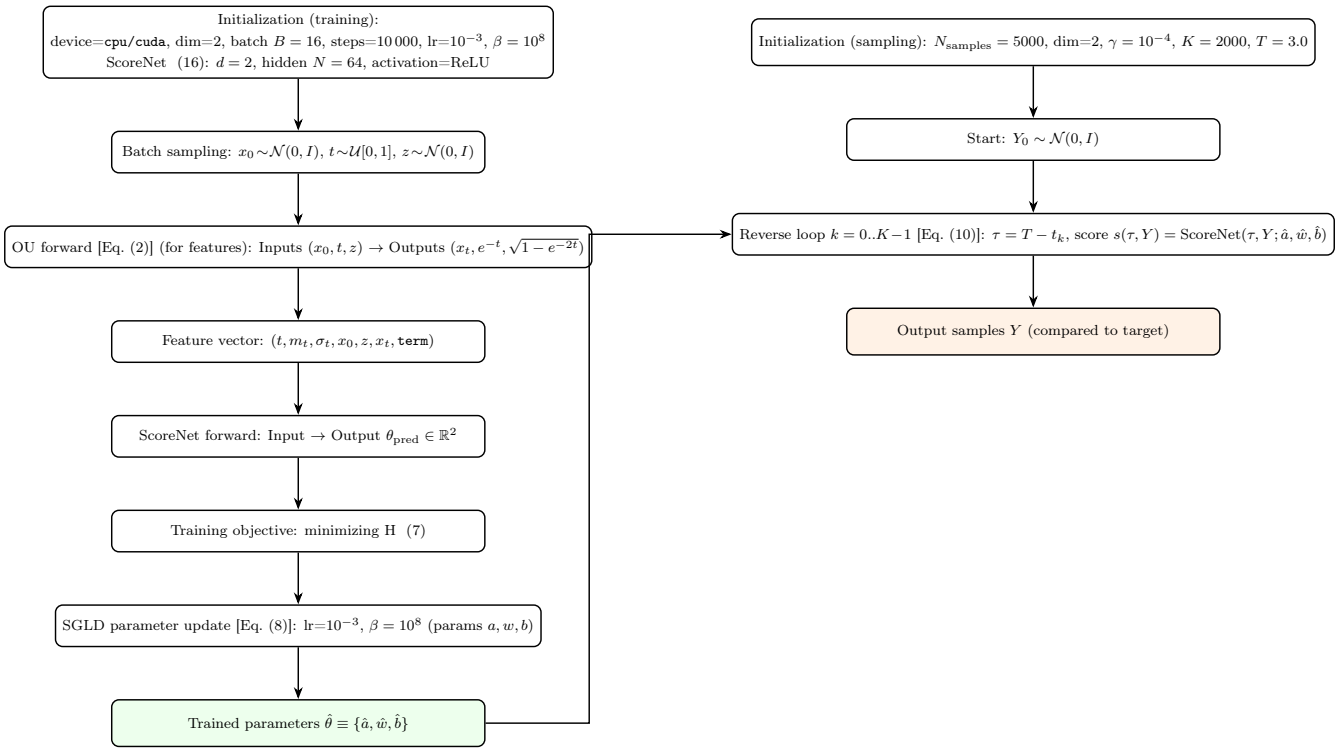
The implementation of the methods and numerical experiments discussed in this dissertation is available in the public GitHub repository: `https://github.com/Pavel-Manyakin/Dissertation`

Below is a list of Jupyter notebooks corresponding to the experiments:

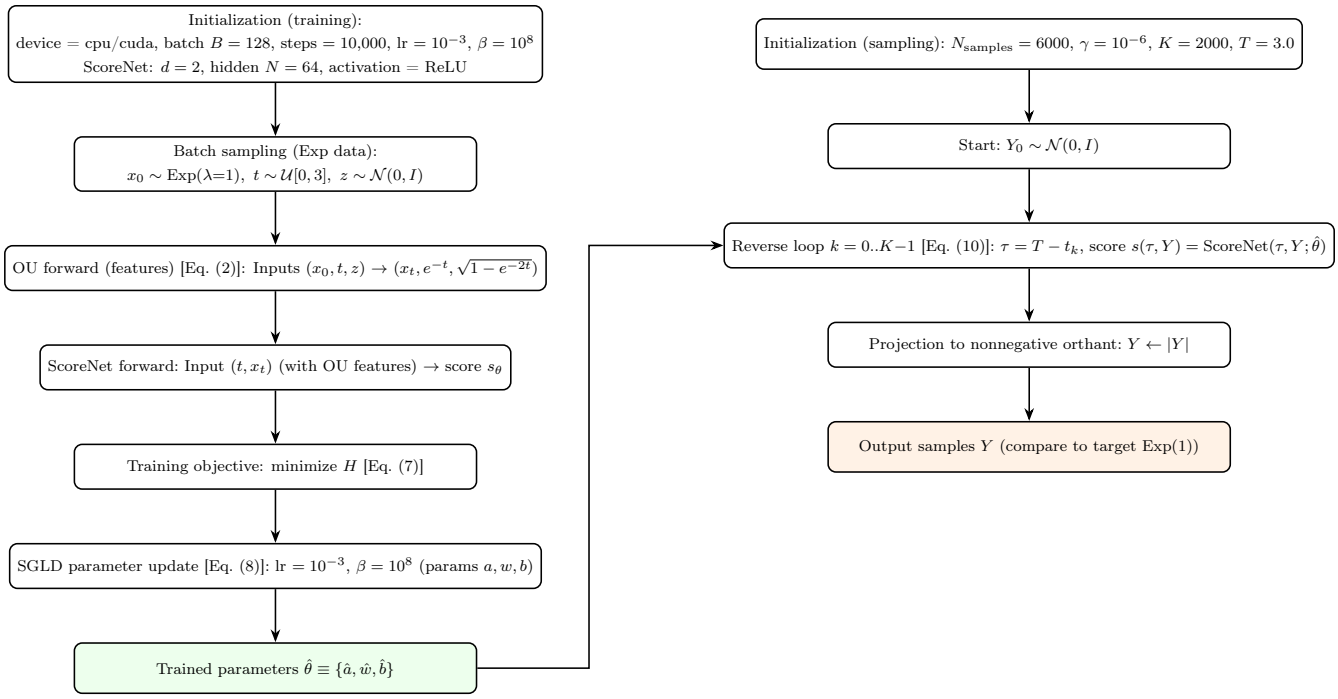
1. Gaussian_Distribution_1D_2.ipynb — experiments with one-dimensional Gaussian distribution



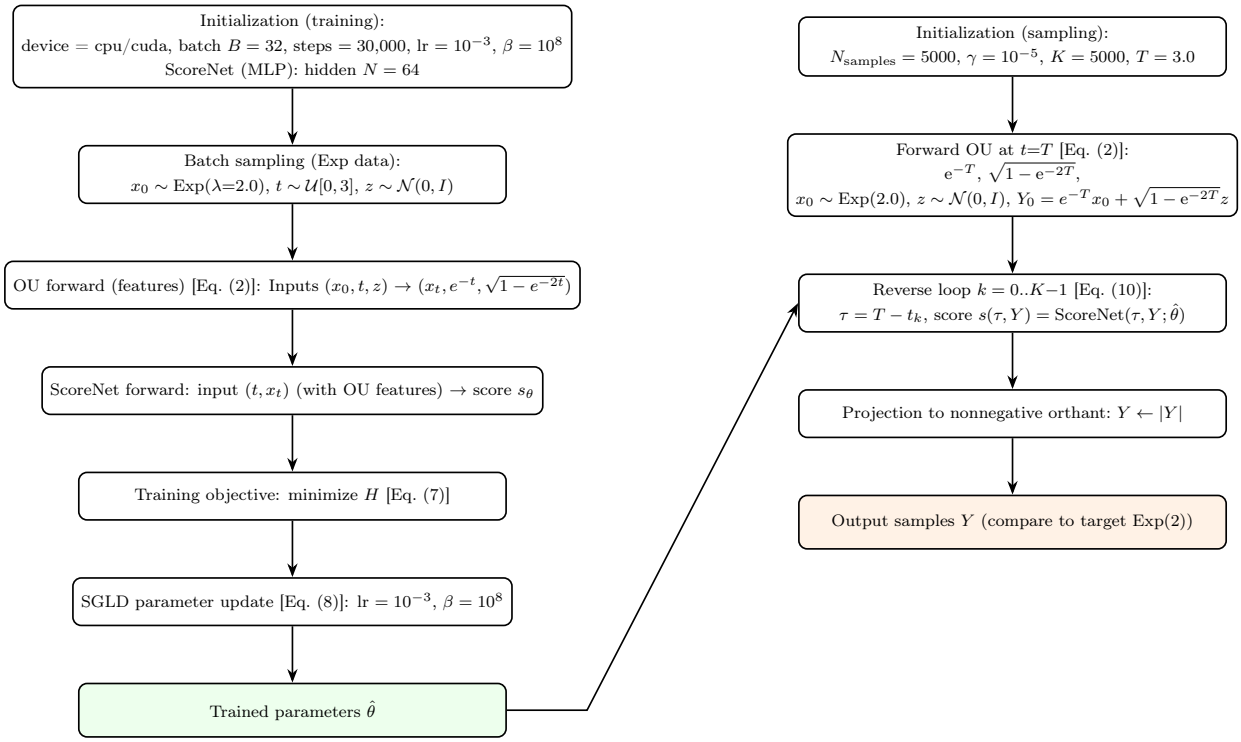
2. Gaussian_2D.ipynb — experiments with two-dimensional Gaussian distribution



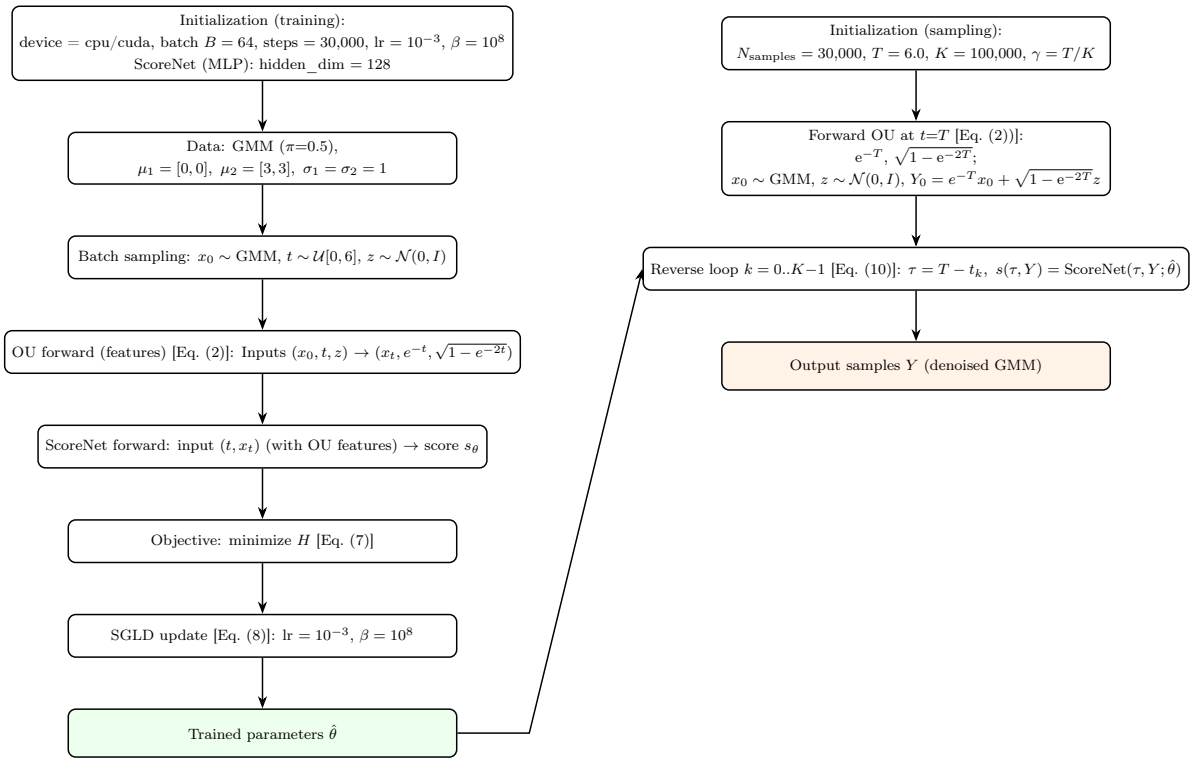

3. Expnential_distribution_1.ipynb — experiments with exponential distribution (part 1)



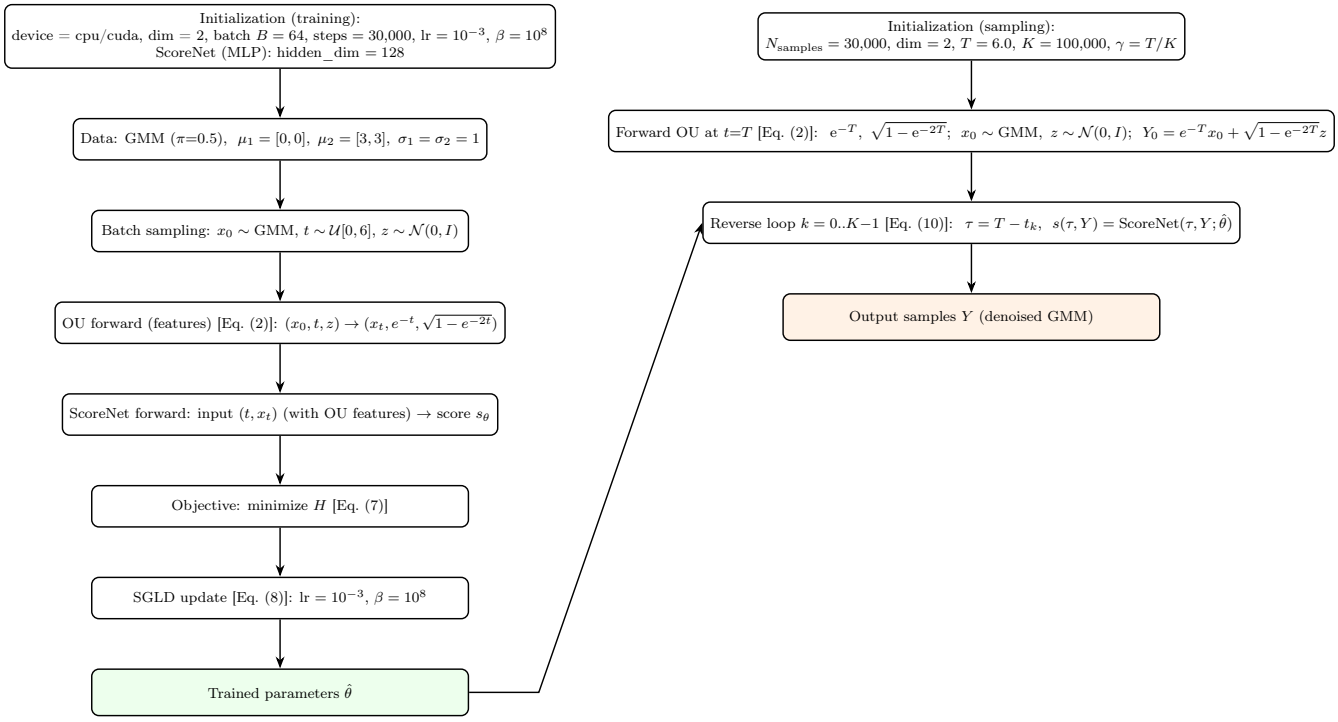
4. Exponential_Distribution-2.ipynb — experiments with exponential distribution (part 2)



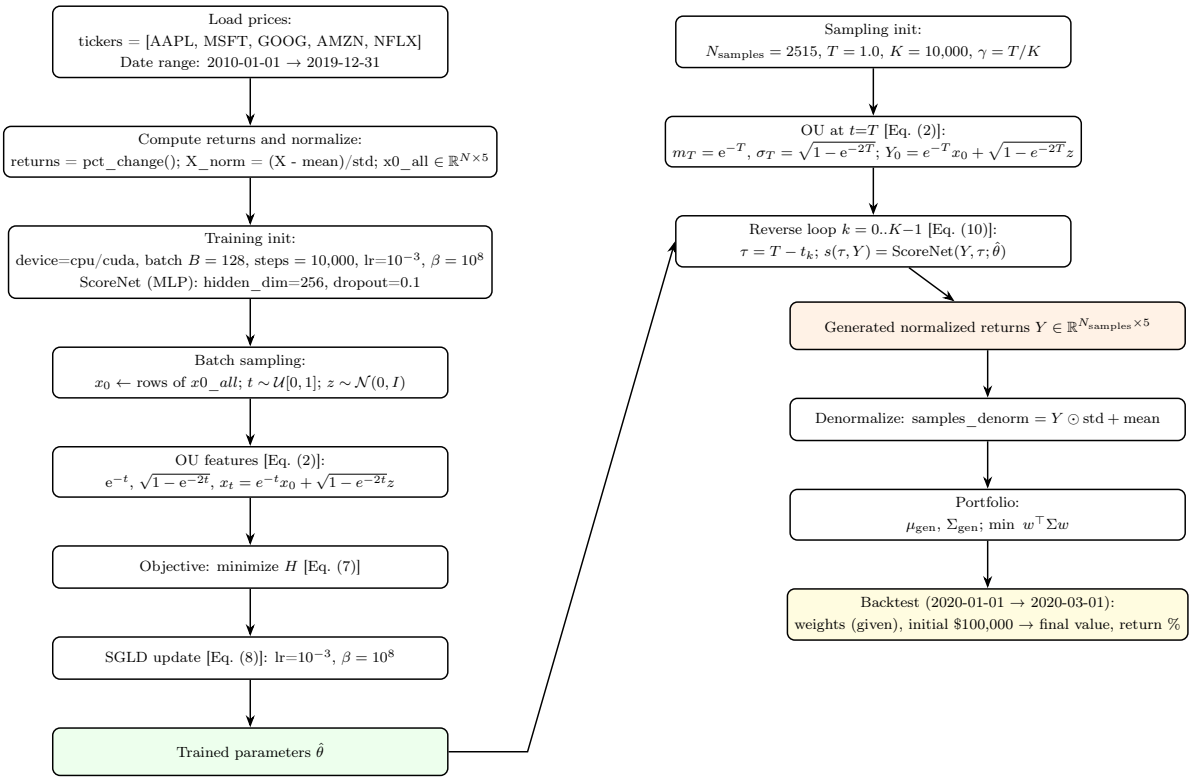

5. Mixed-Gaussian_1D.ipynb — experiments with one-dimensional Gaussian mixture



6. Mixed-Gaussian-2D.ipynb — experiments with two-dimensional Gaussian mixture

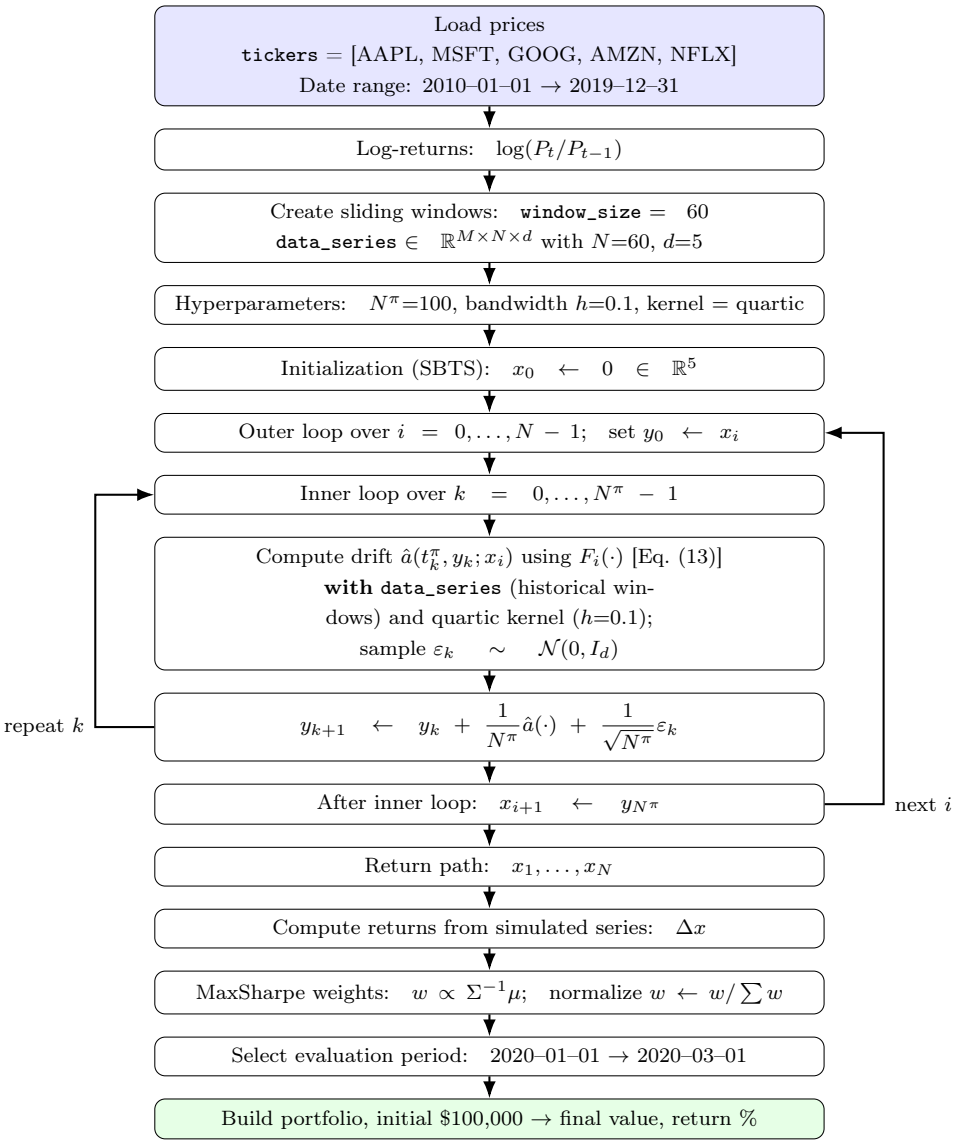



7. Finance_GM_2010-2019.ipynb — financial data experiments (2010–2019) (the algorithm is the same for the 8, 9, 10, 11 but the "Date range" and Backtest period are changing the rest remains the same)



8. Finance_GM_2010-2020_mid.ipynb — financial data experiments (2010–2020, mid-period)
9. Finance_GM_2011-2020.ipynb — financial data experiments (2011–2020)
10. Finance_GM_2011-2021_mid.ipynb — financial data experiments (2011–2021, mid-period)
11. Finance_GM_2012-2021.ipynb — financial data experiments (2012–2021)

12. GM_TS_SHBR_2010-2019.ipynb — Schrödinger bridge experiments (2010–2019) (for 13, 14,15, 16 only "Date range" and evaluation period are changing the rest remains the same)




13. GM_TS_SHBR_2010-2020_mid.ipynb — Schrödinger bridge experiments (2010–2020, mid-period)

14. GM_TS_SHBR_2011-2020.ipynb — Schrödinger bridge experiments (2011–2020)

15. GM_TS_SHBR_2011-2021_mid.ipynb — Schrödinger bridge experiments (2011–2021, mid-period)

16. GM_TS_SHBR_2012-2021.ipynb — Schrödinger bridge experiments (2012–2021)

# References


Benton, J. & V. De Bortoli & A. Doucet & G. Deligiannidis (2024). "Nearly $d$-Linear Convergence Bounds for Diffusion Models via Stochastic Localization". In: *arXiv preprint arXiv:2308.03686*.

Bruno, S. & Y. Zhang & D.-Y. Lim & Ö. D. Akyildiz & S. Sabanis (2025). "On Diffusion-Based Generative Models and Their Error Bounds: The Log-Concave Case with Full Convergence Estimates". In: *arXiv preprint arXiv:2311.13584*.

Chen, G. L. & E. A. Theodorou. (2023). "Likelihood Training of Schrödinger Bridge Using Forward–Backward SDEs Theory". In: *arXiv preprint arXiv:2301.08736*.

Chung, H. & B. Kim & J. C. Kim & M. Kang & J. C. Ye (2024). "Diffusion Posterior Sampling for General Noisy Inverse Problems". In: *arXiv preprint arXiv:2209.14687*.

Conforti, G. & A. Durmus & M. G. Silveri (2024). "KL Convergence Guarantees for Score Diffusion Models under Minimal Data Assumptions". In: *arXiv preprint arXiv:2308.12240*.

De Bortoli, V. & J. Thornton & J. Heng & A. Doucet (2021). "Diffusion Schrödinger Bridge with Applications to Score-Based Generative Modeling". In: *arXiv preprint arXiv:2106.01357*.

Dhariwal, P. & A. Nichol (2021). "Diffusion Models Beat GANs on Image Synthesis". In: *arXiv preprint arXiv:2105.05233*.

Hamdouche, P. H.-L. & H. Pham (2023). "Generative modeling for time series via Schrödinger Bridge". In: *arXiv preprint arXiv:2304.05093*.

Kong, Z. & W. Ping & J. Huang & K. Zhao & B. Catanzaro (2021). "DiffWave: A Versatile Diffusion Model for Audio Synthesis". In: *arXiv preprint arXiv:2009.09761*.

Mei, S. & A. Montanari & P.-M. Nguyen (2018). "A mean field view of the landscape of two-layers neural networks". In: *arXiv preprint arXiv:1804.06561*.

Rombach, R. & A. Blattmann & D. Lorenz & P. Esser & B. Ommer (2022). "High-Resolution Image Synthesis with Latent Diffusion Models". In: *arXiv preprint arXiv:2112.10752*.

Salmona, A. & V. De Bortoli & J. Delon & A. Desolneux (2022). "Can Push-Forward Generative Models Fit Multimodal Distributions?" In: *arXiv preprint arXiv:2206.14476*.

Song, Y. & S. Ermon (2019). "Generative Modeling by Estimating Gradients of the Data Distribution". In: *arXiv preprint arXiv:1907.05600*.

Song, Y. & J. Sohl-Dickstein & D. P. Kingma & A. Kumar & S. Ermon & B. Poole (2021). "Score-Based Generative Modeling through Stochastic Differential Equations". In: *arXiv preprint arXiv:2011.13456*.

Wenliang & B. Moran (2023). "Score-Based generative models learn mainfold-like structures with constrained mixing". In: *arXiv preprint arXiv:2311.09952*.

Yang, K. & A. Wibisono (2023). "Convergence of Score-Based Generative Models under Sub-Gaussian Score Error". In: *arXiv preprint arXiv:2304.12345*.